A project report on

Video-Based Fruit Counting with Tracking to Eliminate Double-Counting

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering Core

by

DERRICK SAMUEL R. (23BCE1040)

PRANNAVAKHANTH (23BCE1869)

PRIYANKA KALIRAJ (23BCE5131)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

A project report on

Video-Based Fruit Counting with Tracking to Eliminate Double-Counting

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering Core

by

DERRICK SAMUEL R. (23BCE1040)

PRANNAVAKHANTH (23BCE1869)

PRIYANKA KALIRAJ (23BCE5131)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

DECLARATION

I hereby declare that the thesis entitled “Video-Based Fruit Counting with Tracking to Eliminate Double-Counting” submitted by DERRICK SAMUEL R. (23BCE1040), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Manas Ranjan Prusty.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai **Signature of the Candidate**

Date:

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"Video-Based Fruit Counting with Tracking to Eliminate Double-Counting"** is prepared and submitted by DERRICK SAMUEL R. (23BCE1040) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering Core** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Manas Ranjan Prusty

Date:

Signature of the Examiner

Name: Dr. Renta Chintala Bhargavi

Date:

Signature of the Examiner

Name: Dr. Ranjith R

Date:

Approved by the Head of Department,
B.Tech. Computer Science and Engineering Core

Signature

Name: **Dr. Prassanna Jayachandran**

Date:

ABSTRACT

OrchardVision is a computer vision system for long-term fruit tracking and counting in mango orchards. It implements identity learning, tracking pipelines, and a full-stack web interface for experiment management and video processing.

Estimating the number of fruit on a tree or across a block is central to yield forecasting, harvest logistics and crop-load management, yet it is still largely done by manual sampling. Counting from orchard video is attractive but difficult: fruit are small, visually similar, frequently hidden by leaves and branches, and the same fruit re-enters the camera view many times as the camera moves along a row or revisits it on later days. A detector-only counter therefore double-counts, and a frame-to-frame tracker alone loses identities whenever a fruit is occluded.

The system developed in this project is built around a processing pipeline that orchestrates frame extraction, object detection, tracking (mock, ByteTrack or TAPNext backends), re-identification embeddings, identity consolidation and analytics. Each fruit is represented by an identity prototype (an embedding vector with quality and observation statistics), so that a fruit that disappears and later reappears can be re-associated with the same identity. Camera motion is estimated and compensated, and tracking quality is reported using IDF1, HOTA and identity-switch metrics. The backend is a FastAPI service with a SQLAlchemy 2.0 data layer (SQLite for development, PostgreSQL for production) and S3-compatible object storage; the frontend is a React and TypeScript application sharing typed contracts with the API. An experiment harness lets users run and compare tracker configurations and export results as JSON or CSV.

The report presents the motivation, related work, requirements, architecture, data model, pipeline design, API design, deployment and testing approach of OrchardVision, followed by an evaluation plan with placeholders for the quantitative results and a discussion of limitations and future work.

Keywords: fruit counting; multi-object tracking; re-identification; identity learning; precision agriculture; mango orchard; FastAPI; React.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Manas Ranjan Prusty, Associate Professor Grade 2, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of computer vision and machine learning.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. Prassanna Jayachandran**, Head of the Department, B.Tech. **Computer Science and Engineering (Core)** and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai **DERRICK SAMUEL R.**

Date:

CONTENTS

CONTENTS iii

LIST OF FIGURES vii

LIST OF TABLES viii

LIST OF ACRONYMS ix

LIST OF SYMBOLS AND NOMENCLATURE x

CHAPTER 1 Introduction 1
- 1.1 Introduction . . . 1
- 1.2 Overview of Orchard Fruit Counting . . . 1
- 1.3 Challenges in Orchard Video . . . 2
- 1.4 Motivation . . . 2
- 1.5 Project Statement . . . 3
- 1.6 Objectives . . . 3
- 1.7 Scope of the Project . . . 4
- 1.8 Organisation of the Report . . . 4

CHAPTER 2 Background and Literature Survey 5
- 2.1 Introduction . . . 5
- 2.2 Fruit Detection in Orchards . . . 5
- 2.3 Tracking-by-Detection . . . 5
  - 2.3.1 SORT and DeepSORT . . . 5
  - 2.3.2 ByteTrack . . . 6
  - 2.3.3 BoT-SORT and Camera Motion Compensation . . . 6
- 2.4 Re-Identification and Identity Learning . . . 6
- 2.5 Tracking Any Point . . . 6
- 2.6 Evaluation Metrics for Tracking . . . 7
- 2.7 Fruit Counting and Yield Estimation from Video . . . 7
- 2.8 Datasets and Benchmarks . . . 7
- 2.9 Learning Objectives for Embeddings . . . 8
- 2.10 Summary of Related Work . . . 8
- 2.11 Comparison of Tracker Families . . . 8
- 2.12 Research Gap . . . 8

CHAPTER 3 System Requirements and Architecture 10
- 3.1 Introduction . . . 10
- 3.2 Requirements . . . 10

3.2.1 Functional Requirements . . . 10
3.2.2 Non-Functional Requirements . . . 10
3.3 High-Level Architecture . . . 11
3.4 Backend . . . 11
3.5 Frontend . . . 12
3.6 Core Services . . . 12
3.7 Configuration . . . 12
3.8 Actors and Use Cases . . . 12
3.9 Hardware and Software Requirements . . . 13
3.10 Technology Stack . . . 13
3.11 Repository Organisation . . . 13

CHAPTER 4 Data Model and Persistence 15
4.1 Introduction . . . 15
4.2 Entities . . . 15
4.3 Entity Relationships . . . 15
4.4 Rows Created by One Processing Run . . . 16
4.5 Identity Records . . . 17
4.6 Tracks and Detections . . . 17
4.7 Telemetry and Camera Motion . . . 17
4.8 Settings and Experiments . . . 17
4.9 Database Selection . . . 17

CHAPTER 5 Processing Pipeline and Identity Learning 19
5.1 Introduction . . . 19
5.2 End-to-End Flow . . . 19
5.2.1 Upload . . . 19
5.2.2 Process . . . 19
5.2.3 Retrieve . . . 20
5.3 Status Models . . . 20
5.4 Processing Stages . . . 21
5.4.1 Frame Extraction . . . 21
5.4.2 Object Detection . . . 21
5.4.3 Tracking . . . 21
5.4.4 Re-Identification Embeddings . . . 22
5.4.5 Identity Consolidation . . . 22
5.4.6 Analytics . . . 23
5.5 Stage Inputs and Outputs . . . 23
5.6 Camera Motion . . . 23
5.7 Identity-Space Visualisation . . . 24
5.8 Identity Consolidation Procedure . . . 24
5.9 Design Considerations . . . 25
5.10 Computational Considerations . . . 25
5.11 Pipeline Parameters . . . 25
5.12 Failure Handling . . . 25
5.13 Reproducibility . . . 26
5.14 Concurrency and Job Control . . . 26

CHAPTER 6 API Design and Implementation 27

6.1 Introduction . . . 27
6.2 Endpoint Summary . . . 27
6.3 Resource-to-Model Mapping . . . 28
6.4 Request Flows . . . 28
6.4.1 Video Upload . . . 28
6.4.2 Starting a Processing Job . . . 30
6.4.3 Reading Results . . . 30
6.4.4 Experiments . . . 30
6.4.5 Settings . . . 31
6.5 Error Handling . . . 31
6.6 Shared TypeScript Types . . . 31
6.7 Frontend Integration . . . 32
6.8 Example Interaction . . . 32

CHAPTER 7 Deployment, Testing and Extension 33
7.1 Introduction . . . 33
7.2 Development Environment . . . 33
7.3 Container Deployment . . . 33
7.4 Configuration . . . 34
7.5 Extending the System . . . 35
7.6 Testing . . . 35
7.7 Security and Robustness Considerations . . . 35
7.8 User Guide . . . 36
7.9 Project Plan . . . 37
7.10 Risk Analysis . . . 37

CHAPTER 8 Experimental Evaluation and Results 39
8.1 Introduction . . . 39
8.2 Experiment Harness . . . 39
8.3 Experimental Set-up . . . 39
8.3.1 Data . . . 39
8.3.2 Configurations . . . 41
8.3.3 Metrics . . . 41
8.4 Detector Selection . . . 41
8.4.1 Experimental Hardware . . . 42
8.4.2 Ground Truth . . . 42
8.5 Tracker Comparison . . . 43
8.6 Effect of Camera Motion Compensation . . . 43
8.7 Identity Prototypes and Re-Identification . . . 45
8.8 Sensitivity to the ReID Threshold . . . 45
8.9 Sensitivity to the Detection Threshold . . . 47
8.10 Counting Accuracy . . . 47
8.11 Runtime Performance . . . 48
8.12 Web Interface . . . 49
8.13 Discussion . . . 49

CHAPTER 9 Conclusion and Future Work 52
9.1 Conclusion . . . 52
9.2 Future Work . . . 52

REFERENCES **54**

APPENDIX 1: REPOSITORY LAYOUT **56**

APPENDIX 2: RUN COMMANDS **57**

APPENDIX 3: ILLUSTRATIVE API RESPONSES **58**

LIST OF FIGURES

3.1 High-level architecture of OrchardVision 11
3.2 Use-case diagram 13

4.1 Overview of the data model 16

5.1 Processing pipeline from upload to persisted results. 20
5.2 Life cycle of a processing job 20

6.1 Sequence diagram of a processing request 29

7.1 Container deployment topology. 34
7.2 User workflow 37

8.1 Sample orchard frames 40
8.2 Detector validation curves 42
8.3 Tracker comparison chart 44
8.4 Camera-motion ablation 44
8.5 Identity embedding projection 45
8.6 Re-identification example 46
8.7 Threshold sweep 47
8.8 Detection confidence sweep 47
8.9 Per-frame processing time 48
8.10 Video list and upload screen 49
8.11 Track visualisation 50
8.12 Experiment management screen 51
8.13 Settings screen 51

LIST OF TABLES

2.1 Summary of related work 9
2.2 Comparison of tracker families 9

3.1 Hardware and software requirements 14
3.2 Technology stack of OrchardVision. 14

4.1 Data models and key fields. 15
4.2 Rows per processed video 16

5.1 Stage inputs and outputs 23
5.2 Key pipeline parameters (defaults stored in `AppSettings`). 26

6.1 REST API endpoints. 27
6.2 Endpoint-to-model mapping 29

7.1 Functional test plan 36
7.2 Project plan 37
7.3 Risk analysis 38

8.1 Evaluation videos 40
8.2 Detector runs 41
8.3 Experimental hardware 42
8.4 Ground-truth construction 43
8.5 Tracker comparison 43
8.6 Camera motion ablation 44
8.7 ReID threshold sensitivity 46
8.8 Counting accuracy 48
8.9 Processing time 48

LIST OF ACRONYMS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
FPS	Frames Per Second
HOTA	Higher Order Tracking Accuracy
IDF1	Identification F1 score
IDSW	Identity Switches (ID switches)
JSON	JavaScript Object Notation
MOT	Multi-Object Tracking
ORM	Object-Relational Mapping
PCA	Principal Component Analysis
ReID	Re-Identification
REST	Representational State Transfer
SQL	Structured Query Language
TAP	Tracking Any Point
t-SNE	t-distributed Stochastic Neighbour Embedding
UI	User Interface
UUID	Universally Unique Identifier

LIST OF SYMBOLS AND NOMENCLATURE

$\mathbf{e}_t$	ReID embedding extracted from a detection at frame t
$\mathbf{p}_k$	Prototype (memory) embedding of identity k
α	Memory update rate used when a prototype is refreshed
τ_{reid}	ReID similarity threshold for accepting a re-association
τ_{conf}	Detection confidence threshold
τ_{match}	Tracker match threshold
IDTP, IDFP, IDFN	Identity-level true positives, false positives and false negatives
HOTA_β	HOTA score at localisation threshold β
DetA, AssA	Detection accuracy and association accuracy components of HOTA

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

OrchardVision is a computer vision system for long-term fruit tracking and counting in mango orchards. It implements identity learning, tracking pipelines, and a full-stack web interface for experiment management and video processing. A user uploads orchard video through the web interface; the backend detects fruit frame by frame, tracks each fruit through time, learns a re-identification (ReID) embedding for it, consolidates the observations into persistent identities and finally computes the analytics of the video. Tracking-quality metrics such as IDF1, HOTA and the number of identity switches are produced by the experiment harness that accompanies the system.

Mango is one of the most widely grown tropical fruit crops, and the timing of harvest, labour planning, packing-house capacity and market commitments all depend on how many fruit an orchard will deliver. Growers traditionally estimate this by hand-counting a small sample of trees and extrapolating to the whole block. The process is slow, subjective and statistically weak, because fruit load varies strongly from tree to tree and from row to row.

Video captured by a handheld camera, a tractor-mounted camera or an unmanned aerial vehicle offers a much denser sample of the orchard. Turning that video into a reliable count, however, is not simply a matter of running an object detector on every frame. Each fruit is visible in dozens or hundreds of consecutive frames, and a detector will report it every time. The central problem is therefore one of *identity*: deciding whether a detection in the current frame is a fruit that has already been counted or a new one.

1.2 OVERVIEW OF ORCHARD FRUIT COUNTING

Automatic fruit counting systems typically combine three ingredients. A detector finds fruit in each frame; a tracker links detections of the same fruit across consecutive frames into short trajectories; and a counting rule converts trajectories into a number, for example by counting the number of unique trajectories or by counting crossings of a virtual line. This design works acceptably when fruit are always visible and the camera moves smoothly in one direction.

Real orchards violate both assumptions. Foliage and branches hide fruit for tens of frames at a time, which breaks trajectories and inflates the count when the fruit reappears. In long-term monitoring the same tree is recorded on several days, or the camera returns to a row after a turn, so identities must survive gaps of seconds, minutes or days. OrchardVision addresses this by adding an *identity learning* layer on top of tracking: every identity is represented by a prototype embedding that is updated as new observations arrive, and tracks that end and later reappear can be linked back to the same identity through a ReID step.

The system is organised as a pipeline (frame extraction, detection, tracking, ReID embedding, identity consolidation, analytics), a REST API that exposes every intermediate and final result, a relational data model that persists them, and a web client with an experiment harness for comparing configurations.

1.3 CHALLENGES IN ORCHARD VIDEO

The following factors make fruit tracking in orchards considerably harder than the pedestrian and vehicle tracking problems on which most multi-object tracking research has been developed:

- **Occlusion.** Leaves, branches and other fruit frequently hide a fruit partially or completely, so tracks fragment.
- **Similar appearance.** Fruit on the same tree share colour, shape and texture, which weakens appearance cues that ReID relies on.
- **Camera motion.** Handheld or vehicle-mounted cameras move constantly, so image-space motion models (such as constant-velocity Kalman filters) are disturbed by ego-motion rather than by object motion.
- **Illumination and scale changes.** Sunlight, shadows and varying distance to the canopy change the apparent size and colour of fruit.
- **Long gaps.** In long-term monitoring an identity may disappear for far longer than any tracker buffer, so matching must rely on learned appearance rather than short-term motion.
- **Dense small objects.** A single frame may contain tens of small, closely spaced fruit, increasing the risk of identity switches.

1.4 MOTIVATION

Three observations motivate the project. First, yield estimates are needed early and repeatedly during the season, and manual sampling cannot keep up with this demand

at the scale of commercial orchards. Second, video is cheap to collect and is already captured during routine orchard operations, so a counting method that runs on ordinary video is easy to adopt. Third, counting errors from video are dominated by identity errors rather than detection errors, which means that progress requires explicit support for identity: storing what each fruit looks like, remembering it after it disappears and measuring how well this works.

A further motivation is engineering. Research prototypes in this area are often a script that prints a number. A reusable system needs persistent results, a stable API, reproducible experiments and a user interface, so that agronomists and researchers can examine and trust the output. OrchardVision therefore treats the data model, the API and the experiment harness as first-class parts of the contribution.

1.5 PROJECT STATEMENT

To design and implement an end-to-end, web-based system that ingests orchard video, detects and tracks individual mango fruit, maintains a persistent learned identity for each fruit across occlusions and re-appearances, and reports tracking quality with standard metrics, while making every stage configurable and every result inspectable and comparable through a reproducible experiment harness.

1.6 OBJECTIVES

The objectives of the project are:

1. To implement a processing pipeline that orchestrates detection, tracking, camera-motion compensation, ReID embedding, identity consolidation and analytics for an uploaded video.
2. To support multiple tracker back-ends (mock, ByteTrack and TAPNext) behind a common interface so that they can be compared on the same video.
3. To learn and persist an identity prototype (embedding vector, quality and observation count) for every fruit and to record ReID events when an identity is lost and later reappears.
4. To estimate camera motion and record whether compensation was applied.
5. To compute IDF1, HOTA, ID-switch, fragmentation, count-error and identity-recovery metrics, together with per-frame timing telemetry.
6. To expose all results through a documented REST API and a React / TypeScript web client with shared type definitions.

7. To provide an experiment harness that runs and stores tracker evaluations and exports them as JSON or CSV.
8. To make the system deployable on a developer laptop and as a container stack (API, queue worker and broker) that runs the same code in both settings.
9. To keep every intermediate result inspectable: detections, tracks, identities, observations, re-identification events and timings are persisted separately so that a counting error can be traced to the stage that produced it.

1.7 SCOPE OF THE PROJECT

The scope of the work covers the backend service, the data model, the processing pipeline, the REST API, the shared TypeScript types, the deployment configuration and the experiment-management interface described in the following chapters. Uploads are limited to 2 GB per video by default (configurable through the `MAX_UPLOAD_BYTES` environment variable).

The following are explicitly outside the scope of this report: training new fruit detectors from scratch, fruit-size or ripeness estimation, three-dimensional orchard mapping, and multi-camera fusion. The evaluation reported in Chapter 8 is limited to the videos and tracker configurations listed there.

1.8 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature on fruit detection, multi-object tracking, re-identification and tracking metrics. Chapter 3 presents the requirements and the system architecture. Chapter 4 describes the data model. Chapter 5 explains the processing pipeline and the identity-learning design. Chapter 6 documents the REST API, error handling and shared types. Chapter 7 covers deployment and testing. Chapter 8 presents the evaluation set-up and results, and Chapter 9 concludes the report and outlines future work. References and appendices follow.

CHAPTER 2

BACKGROUND AND LITERATURE SURVEY

2.1 INTRODUCTION

This chapter reviews the techniques on which OrchardVision builds: deep-learning-based fruit detection, tracking-by-detection, appearance-based re-identification, point tracking, camera-motion compensation and the metrics used to judge tracking quality. The chapter closes with a comparison table and the research gap that motivates the project.

2.2 FRUIT DETECTION IN ORCHARDS

Deep convolutional detectors have replaced hand-crafted colour and shape features for fruit detection. Bargoti and Underwood (2017) [2] showed that a Faster R-CNN detector trained on a few hundred annotated images could detect mangoes, apples and almonds in orchard imagery with high F1 scores, and Koirala et al. (2019) [9] surveyed deep-learning methods for fruit detection and yield estimation, and report that single-stage detectors from the YOLO family (Redmon et al., 2016) [14] have become the common choice for field deployment because of their accuracy and speed. The YOLO implementation used here is the maintained Ultralytics package [23]. In OrchardVision the detector is a pluggable component: the model is selected through a `model_path` setting, and detections are filtered by a configurable confidence threshold.

2.3 TRACKING-BY-DETECTION

2.3.1 SORT AND DEEPSORT

SORT (Bewley et al., 2016) [5] links detections across frames with a Kalman filter (Kalman, 1960) [8] for motion prediction and the Hungarian algorithm (Kuhn, 1955) [10] for assignment on an intersection-over-union cost. It is fast but suffers many identity switches under occlusion. DeepSORT (Wojke et al., 2017) [16] adds a learned appearance embedding to the association cost, which substantially reduces identity switches at the price of an extra network evaluation per detection.

2.3.2 BYTETRACK

ByteTrack (Zhang et al., 2022) [17] observes that low-confidence detections are often occluded objects rather than noise. It associates high-confidence detections first and then uses the low-confidence ones to recover tracks that would otherwise be lost. This is particularly relevant for fruit, where partial occlusion lowers detector confidence. ByteTrack is one of the tracker back-ends supported by OrchardVision.

2.3.3 BOT-SORT AND CAMERA MOTION COMPENSATION

BoT-SORT (Aharon et al., 2022) [1] combines motion and appearance information and introduces camera-motion compensation, in which the global image motion between frames is estimated and used to correct the Kalman-filter predictions. Because orchard video is almost always recorded from a moving platform, compensating for camera motion is expected to matter more than in fixed-camera surveillance. OrchardVision records the estimated motion type and whether compensation was applied for every processed video.

2.4 RE-IDENTIFICATION AND IDENTITY LEARNING

Re-identification originates in person re-identification, where a network maps an image crop to an embedding such that crops of the same identity are close and crops of different identities are far apart. Hermans et al. (2017) [7] showed that a carefully mined triplet loss is a strong baseline for this task. For tracking, an identity can be summarised by a *prototype* embedding obtained by aggregating the embeddings of its observations; new detections are compared with prototypes, and a similarity above a threshold triggers re-association. This is the approach followed by OrchardVision: each identity stores a prototype vector, a quality score, an observation count and a flag indicating whether the identity is confident.

2.5 TRACKING ANY POINT

Point-tracking methods follow an arbitrary query point through a video, even through occlusion. TAP-Vid (Doersch et al., 2022) [6] introduced a benchmark for this task, and TAPNext (Zholus et al., 2025) [18] formulates point tracking as next-token prediction in a sequence model. Because fruit are small and lack distinctive texture, tracking a point on the fruit surface can be more stable under occlusion than tracking a bounding box. TAPNext is available in OrchardVision as one of the tracker options.

2.6 EVALUATION METRICS FOR TRACKING

Three families of metrics are used in this work. The CLEAR MOT metrics (Bernardin and Stiefelhagen, 2008) [4] count identity switches (IDSW), false positives and misses. IDF1 (Ristani et al., 2016) [15] measures how long identities are correctly maintained and is defined as

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}} \tag{2.1}$$

where IDTP, IDFP and IDFN are identity-level true positives, false positives and false negatives obtained from a global one-to-one matching between ground-truth and predicted trajectories. HOTA (Luiten et al., 2021) [12] balances detection and association quality:

$$\mathrm{HOTA}_\beta = \sqrt{\mathrm{DetA}_\beta \cdot \mathrm{AssA}_\beta} \tag{2.2}$$

and is usually reported as the average over a range of localisation thresholds β. For fruit counting, IDF1 and the number of identity switches are especially informative, because every identity switch changes the count.

2.7 FRUIT COUNTING AND YIELD ESTIMATION FROM VIDEO

Liu et al. (2019) [11] proposed monocular-camera fruit counting with semantic data association, where fruit are tracked across frames and associated with a 3-D map to avoid double counting. Their work demonstrates the value of explicit data association but relies on structure-from-motion, which is expensive and brittle in foliage-rich scenes. OrchardVision pursues a complementary, appearance-based route: identities are maintained by learned embeddings and can be evaluated through the same experiment harness as any other tracker.

2.8 DATASETS AND BENCHMARKS

Tracking research is usually benchmarked on pedestrian and vehicle datasets, in which the metrics above were defined, whereas orchard datasets for tracking are scarce. Point-tracking has its own benchmark in TAP-Vid (Doersch et al., 2022) [6]. Because public orchard-tracking benchmarks with persistent fruit identities are limited, a practical evaluation of OrchardVision requires manually annotated orchard videos in which each fruit is assigned a persistent identifier. The experiment harness of the system is

designed to run the same configurations on any such video and to export the metrics for comparison.

2.9 LEARNING OBJECTIVES FOR EMBEDDINGS

Embedding networks for re-identification are commonly trained with classification losses, with metric-learning losses such as the triplet loss (Hermans et al., 2017) [7], or with a combination of both. For fruit, the main difficulty is that different instances look alike, so metric-learning losses with hard-example mining are attractive: they explicitly push embeddings of different fruit apart and pull embeddings of the same fruit together across viewpoints and lighting. The quality of the embedding determines how reliably identity matching can bridge long gaps, which is why the identity prototype in OrchardVision stores a quality score and an observation count along with the embedding.

The implementation follows this pattern. Crop pairs mined from annotated tracklets are trained with a contrastive InfoNCE loss (Reddi et al., 2017) [22] on top of the triplet baseline, and pseudo-labels for unlabelled tracklets are generated by a density-based clustering step in the style of DBSCAN (Ester et al., 1996) [19]. The resulting embeddings are inspected in two dimensions with t-SNE (van der Maaten and Hinton, 2008) [21], falling back to a linear principal-component projection (Jolliffe and Cadima, 2016) [20] when the number of identities is small.

2.10 SUMMARY OF RELATED WORK

2.11 COMPARISON OF TRACKER FAMILIES

2.12 RESEARCH GAP

Existing fruit-counting work either focuses on single-frame detection or on short-term tracking, and rarely exposes the intermediate artefacts (tracks, identity prototypes, ReID events, per-frame timing, camera motion) needed to understand *why* a count is wrong. There is also little tooling for repeatable comparison of tracker configurations on the same orchard video. OrchardVision addresses both gaps by combining identity learning with a persistent, queryable data model and an experiment harness.

Work	Technique	Strength	Limitation
Bargoti & Underwood (2017)	Faster R-CNN fruit detection	Accurate per-frame detection	No temporal identity
Bewley et al. (2016)	SORT: Kalman + Hungarian	Very fast	Many ID switches under occlusion
Wojke et al. (2017)	DeepSORT: adds appearance embedding	Fewer ID switches	Extra embedding cost; limited long gaps
Zhang et al. (2022)	ByteTrack: uses low-score boxes	Recovers occluded objects	Motion-based; no learned identity memory
Aharon et al. (2022)	BoT-SORT: motion + appearance + camera compensation	Robust to ego-motion	Designed for pedestrians
Liu et al. (2019)	Fruit counting with semantic data association	Avoids double counting through 3-D map	Requires structure from motion
Zholus et al. (2025)	TAPNext point tracking	Tracks through occlusion	Point-based; needs query points

Table 2.1: Summary of related work on detection, tracking and fruit counting.

Family	Association cue	Handles long occlusion	Typical cost
Motion only (SORT, ByteTrack)	Kalman prediction and IoU	Poorly beyond the track buffer	Low
Motion + appearance (DeepSORT, BoT-SORT)	Embedding distance combined with motion	Moderately	Medium (embedding per detection)
Point tracking (TAPNext)	Point trajectories through the video	Well, if the point remains trackable	Higher (sequence model)
Identity memory (this work)	Prototype embeddings with ReID threshold	Designed for long gaps	Medium (embedding and matching)

Table 2.2: Qualitative comparison of tracker families.

CHAPTER 3

SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 INTRODUCTION

OrchardVision is a full-stack system consisting of a FastAPI backend, a React + TypeScript frontend and a set of core services that implement the computer vision pipeline. This chapter states the requirements that the system satisfies and describes its architecture and technology stack.

3.2 REQUIREMENTS

3.2.1 FUNCTIONAL REQUIREMENTS

The functional requirements below were derived from the capabilities of the implemented system:

- FR1: upload a video, store it, and extract its metadata (width, height, frame rate and frame count).
- FR2: start, monitor and complete a processing job for an uploaded video, and reject a new job while another is active.
- FR3: persist tracks, detections, identities, ReID events, camera-motion estimates and per-frame timing for each processed video.
- FR4: retrieve every persisted artefact through the REST API, including a two-dimensional projection of identity prototypes.
- FR5: create, run, list, export and delete evaluation experiments for different trackers and scenarios.
- FR6: read and update the default pipeline settings.
- FR7: delete a video together with all associated data.

3.2.2 NON-FUNCTIONAL REQUIREMENTS

- **Portability.** The same code runs on a developer laptop and in the container stack; because all persistence goes through the SQLAlchemy ORM, the database is selected purely by the connection string in `DATABASE_URL`.

- **Responsiveness.** Long-running processing must not block API requests; it is executed as a background task or through a Celery queue.
- **Consistency.** All errors use a single JSON structure and the frontend and backend share type definitions.
- **Extensibility.** New models, routers, schemas and services can be added in separate modules without modifying existing ones.
- **Isolation of binary data.** Videos and cached frames are written to the file system through the storage service rather than into the database, so large payloads never travel through SQL.

3.3 HIGH-LEVEL ARCHITECTURE

Figure 3.1 shows the components of OrchardVision and their interaction. The React client talks to the FastAPI backend over REST under the `/api` prefix; the backend uses the core services to process videos, persists results through SQLAlchemy and writes binary data to the file system.

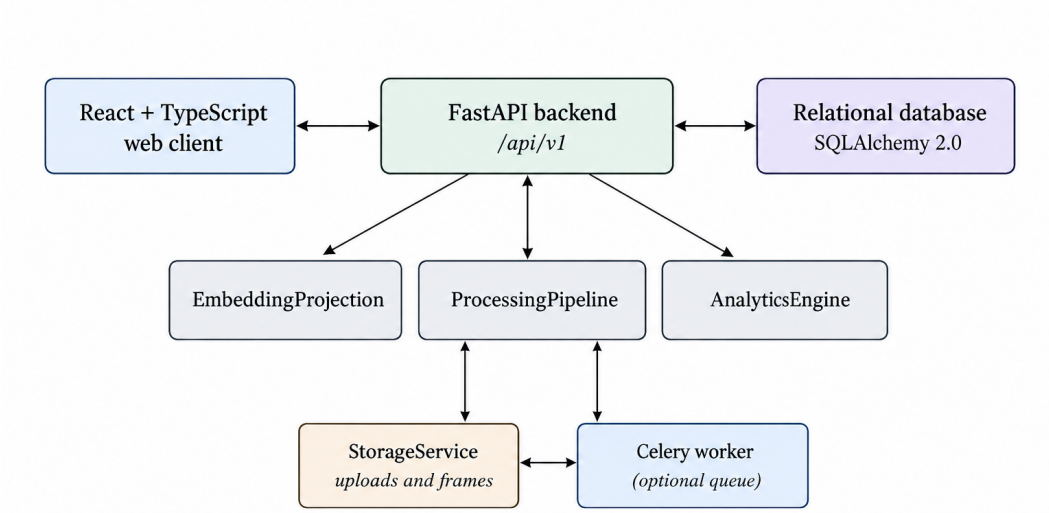


Figure 3.1: High-level architecture of OrchardVision showing the web client, FastAPI backend, core services, relational database, file storage and optional Celery worker.

3.4 BACKEND

The backend is implemented with the following choices:

- **Framework:** FastAPI [13] with the SQLAlchemy 2.0 ORM [3].
- **Database:** SQLite, selected through the `DATABASE_URL` environment variable; the ORM layer allows a server database to be substituted without code changes.

- **API:** REST under `/api/v1/`, organised in router prefixes for health (`GET /health`), videos (`POST/GET/DELETE /videos`), tracks (`GET /videos/{id}/tracks`), identities (`GET /videos/{id}/identities`), processing (`POST /videos/{id}/process`), jobs (`GET /jobs/{id}`), experiments (`GET/POST/DELETE /experiments`) and settings (`GET/PUT /settings`).

3.5 FRONTEND

The frontend is a React and TypeScript application. Types shared with the API are defined once in `packages/types/src/`, the components live in `apps/web/src/`, and an API client calls the FastAPI backend. During development the Vite server proxies requests under `/api/` to the backend.

3.6 CORE SERVICES

- **ProcessingPipeline.** Orchestrates frame extraction → detection → tracking → embedding → analytics.
- **EmbeddingProjection.** Produces t-SNE / PCA visualisations of identity prototypes, served as two-dimensional coordinates.
- **AnalyticsEngine.** Computes IDF1, HOTA and ID-switch metrics.
- **StorageService.** Saves uploads under `UPLOAD_DIR` and renders cached JPEG frames under `UPLOAD_DIR/frames/<video\_id>`, validating that resolved paths stay inside the upload directory.

3.7 CONFIGURATION

Configuration is loaded through `pydantic_settings` from an `.env` file. The key settings are the confidence threshold, the ReID threshold, the tracker type, the camera-motion mode, the memory update rate, the tracker match threshold and the number of buffer frames. The defaults are persisted in the `AppSettings` table, which holds a single row identified by `GLOBAL_SETTINGS_ID`, and can be read and updated through `GET /settings` and `PUT /settings`.

3.8 ACTORS AND USE CASES

Two kinds of users interact with the system. An *analyst* (for example an agronomist or a student researcher) uploads videos, starts processing, inspects tracks and identities and reads analytics. An *experimenter* additionally creates and compares experiments and edits the pipeline defaults. The main use cases are: upload video, process video, view

tracks, view identities and their projection, view ReID events, view analytics, manage experiments, export results, edit settings and delete video.

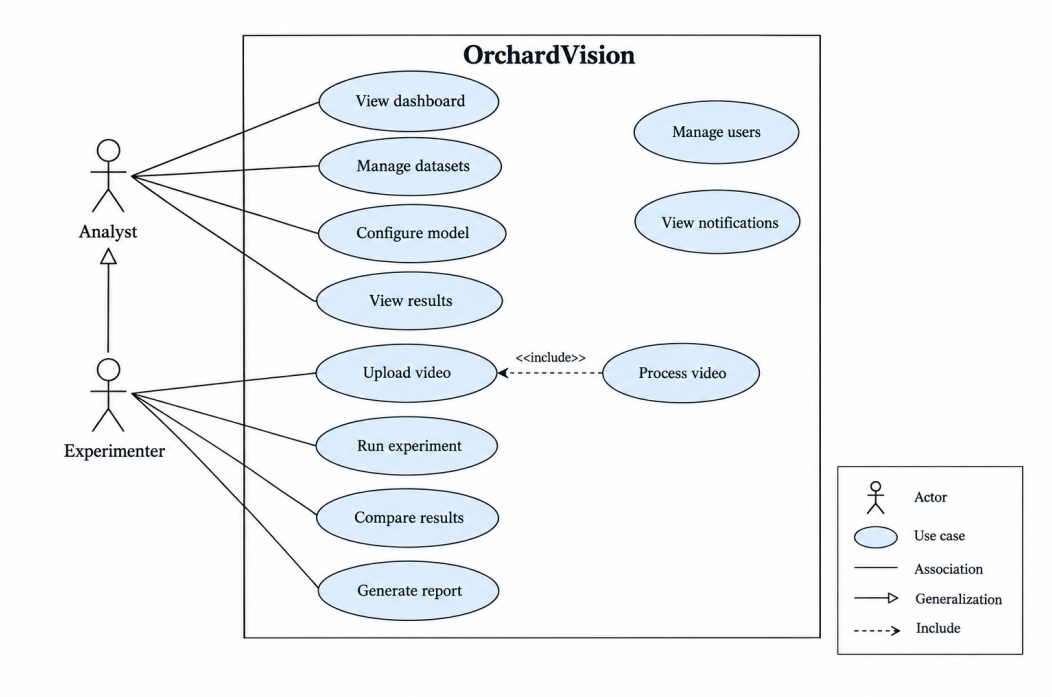


Figure 3.2: Use-case diagram of OrchardVision.

3.9 HARDWARE AND SOFTWARE REQUIREMENTS

3.10 TECHNOLOGY STACK

3.11 REPOSITORY ORGANISATION

New models and migrations are added as SQLAlchemy 2.0 model definitions. New API routes live in `apps/api/app/routers/`, schemas in `apps/api/app/schemas/` and services in `apps/api/app/services/`. The application entry point and the global exception handler are in `apps/api/app/main.py`. The complete layout is given in Appendix A.

Item	Requirement
Operating system	Linux, macOS or Windows capable of running Python, Node.js and Docker
Python	Python environment with FastAPI, SQLAlchemy 2.0, pydantic-settings, uvicorn, Celery, OpenCV and the detection/tracking libraries used by the pipeline
Node.js	Node.js with npm and pnpm for the frontend and the shared types package
Database	SQLite
File storage	Local disk through the storage service
Compute	A machine able to run the detector and the trackers; the experiments reported in Chapter 8 were run on an Apple M2 Max (12 CPU cores, 30 GPU cores, 64 GB unified memory) with PyTorch 2.14 and Ultralytics 8.4
Browser	A current desktop browser (the frontend is served by Vite in development)

Table 3.1: Hardware and software requirements.

Layer	Technology
Web framework	FastAPI served by uvicorn (gunicorn/uvicorn in production)
ORM / database	SQLAlchemy 2.0; SQLite (substitutable through `DATABASE_URL`)
Settings	`pydantic_settings` loading `.env`
Background work	FastAPI background task, or Celery when a queue is configured
File storage	Local disk (uploads and frame cache)
Frontend	React, TypeScript, Vite
Shared contracts	TypeScript package in `packages/types/src/`
Testing / checks	`pytest`; `pnpm exec tsc --noEmit`

Table 3.2: Technology stack of OrchardVision.

CHAPTER 4

DATA MODEL AND PERSISTENCE

4.1 INTRODUCTION

All results produced by the pipeline are persisted so that they can be retrieved later through the API without reprocessing the video. The data layer is defined with SQLAlchemy 2.0 model definitions. Most tables carry a `video_id` that links them to the `Video` row they were derived from, which is why deleting a video (`DELETE /videos/{id}`) removes all associated data.

4.2 ENTITIES

Table 4.1 lists the persisted models and their key fields.

Table 4.1: Data models and key fields.

Model	Key fields
`Video`	`id`, `filename`, `width`, `height`, `fps`, `frame_count`, `status`
`ProcessingJob`	`id`, `video_id`, `status`, `stage`, `progress`, `config` (dict)
`Experiment`	`id`, `name`, `tracker`, `scenario`, `num_frames`, `status`, `metrics`
`IdentityRecord`	`id`, `video_id`, `identity_id`, `prototype` (embedding vector), `quality`, `observation_count`, `is_confident`
`Track`	`id`, `video_id`, `track_id`, `frames`, `first_seen`, `last_seen`
`DetectionRecord`	`id`, `video_id`, `frame_id`, `bbox`, `confidence`
`FrameTimingRecord`	`id`, `video_id`, `frame_id`, `proc_ms`
`CameraMotionRecord`	`id`, `video_id`, `motion_type`, `compensation_applied`
`IdentityObservation`	`id`, `video_id`, `identity_id`, `track_id`, `frame_id`
`ReidEvent`	`id`, `video_id`, `lost_frame_id`, `reappearance_frame_id`
`AppSettings`	`id` (=`GLOBAL_SETTINGS_ID`), `values` (dict)

4.3 ENTITY RELATIONSHIPS

Figure 4.1 gives an overview of how the models relate to each other. The relationships shown are inferred from the `video_id`, `identity_id`, `track_id` and `frame_id` fields listed above; `Experiment` and `AppSettings` are independent of any single video.

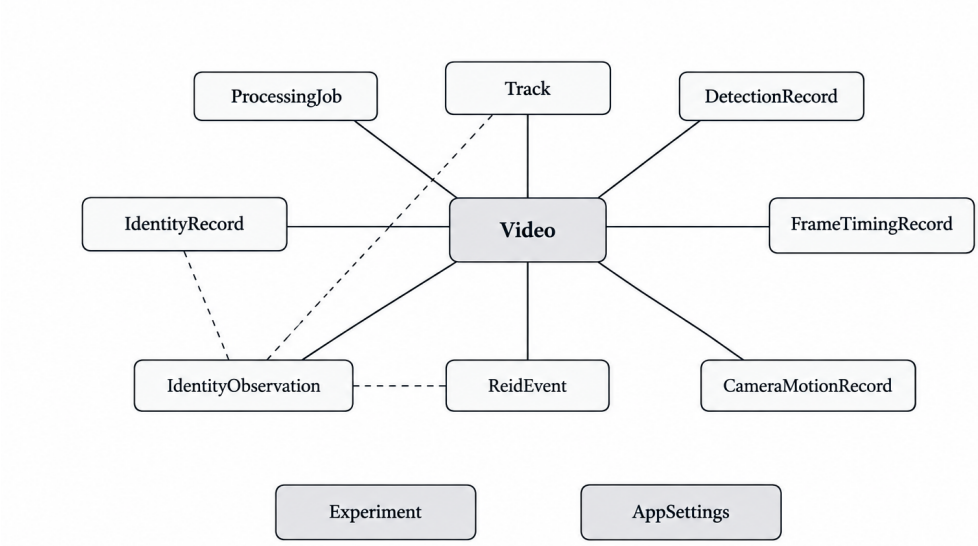


Figure 4.1: Overview of the data model: all per-video artefacts reference `Video`; `Experiment` and `AppSettings` are standalone.

4.4 ROWS CREATED BY ONE PROCESSING RUN

When a processing job completes, the `Video` status becomes `processed` and the following rows are created: `IdentityRecord`, `Track`, `DetectionRecord`, `FrameTimingRecord` and `CameraMotionRecord`; identity sightings and re-identification events are stored as `IdentityObservation` and `ReidEvent` rows. With F processed frames, $\bar{d}$ mean detections per frame, K tracklets, U distinct identities, s_i sightings of identity i, E occlusion or re-appearance events and M motion estimate rows, Table 4.2 reports the volume of data produced by one real run: $F = 141$, $\bar{d} = 5.65$, $K = 7$, $U = 15$, $\sum_i s_i = 714$, $E = 0$ and $M = 1$.

Model	Rows per video	Growth driver
`DetectionRecord`	796	Frames $\times$ detections per frame
`FrameTimingRecord`	141	One per processed frame
`Track`	7	Number of tracks started
`IdentityRecord`	15	Number of distinct fruit
`IdentityObservation`	714	Sightings of identities
`ReidEvent`	0	Occlusion/re-appearance events
`CameraMotionRecord`	1	Motion estimate(s) for the video

Table 4.2: Rows created by one real-mode processing run of V2 (`videos/test4.mp4`, 141 frames, 5.9 s at 13.1 tracking FPS). The third column gives the symbolic growth driver for each count.

4.5 IDENTITY RECORDS

An `IdentityRecord` stores the learned prototype of one fruit: an embedding vector, a quality score, the number of observations that contributed to it, and an `is_confident` flag. Each individual sighting of an identity in a frame, attributed to a track, is stored as an `IdentityObservation`. A `ReidEvent` records the frame in which an identity was lost and the frame in which it reappeared.

This split keeps the compact summary (the prototype) separate from the evidence that produced it (the observations), which allows the interface to show both the identity-space visualisation and the per-frame history of any identity.

4.6 TRACKS AND DETECTIONS

A `Track` holds the frames over which a tracker followed an object, together with its first and last frame. `DetectionRecord` stores the bounding box and confidence of each persisted detection. In the shared TypeScript types the bounding box is a `BoundingBox` with `x`, `y`, `width` and `height`, and a track point can be in one of three visibility states: `visible`, `occluded` or `lost`.

4.7 TELEMETRY AND CAMERA MOTION

`FrameTimingRecord` stores the processing time per frame in milliseconds (`proc_ms`) and is returned by `GET /videos/{id}/per-frame`. `CameraMotionRecord` stores the type of motion estimated for the video and whether compensation was applied, and is returned by `GET /videos/{id}/motion`.

4.8 SETTINGS AND EXPERIMENTS

`AppSettings` is a single-row table whose `values` dictionary holds the pipeline defaults. `Experiment` stores the name, tracker, scenario, number of frames, status and computed metrics of an evaluation run. An experiment that is in the `RUNNING` state cannot be deleted.

4.9 DATABASE SELECTION

SQLite is used throughout, with the development database file at `data/orchardvision.db` and the container stack pointing `DATABASE_URL` at a file in a shared volume. Because all access goes through the SQLAlchemy ORM, the application code is identical whichever server is selected; only the connection string changes. The current limits

of SQLite become visible with several concurrent writers, which is the main reason a deployment at scale would move to a server database. Every identifier is a 32-character hexadecimal UUID, and only the video-to-job relation is mapped as an ORM relationship; the remaining foreign keys are plain indexed columns, which keeps the persistence of high-volume tables such as detections and frame timings cheap.

CHAPTER 5

PROCESSING PIPELINE AND IDENTITY LEARNING

5.1 INTRODUCTION

This chapter explains how an uploaded video becomes a set of tracks, identities and analytics. It first describes the end-to-end flow (upload, process, retrieve), then each processing stage, the state models that describe progress, and the configuration parameters that control behaviour.

5.2 END-TO-END FLOW

5.2.1 UPLOAD

The user posts a video to `POST /videos`. The backend saves the file through `storage_service.save_upload`, extracts its metadata with `extract_video_metadata`, and creates a `Video` row with status `uploaded`.

5.2.2 PROCESS

A request to `POST /videos/{id}/process` first validates that there is no active job for the video. It then creates a `ProcessingJob` with status `queued`, and either enqueues the work (Celery, when a broker is configured) or runs it synchronously as a background task. Both paths execute the same pipeline in `app/services/processing.py`, which performs:

1. frame extraction and object detection in a single streaming pass over the video;
2. tracking, with camera-motion compensation applied before association;
3. re-identification embeddings and identity consolidation;
4. analytics computation (`compute_video_analytics`).

The detector and the embedding extractor are chosen by the inference mode. In `mock` mode a deterministic synthetic detector produces detections for the orchard scenario, which is what makes the development and test runs reproducible. In `real` mode the pipeline uses a YOLO detector loaded from `YOLO_WEIGHTS_PATH` and, for the embeddings, either the checkpoint at `EMBEDDING_MODEL_PATH` or a torchvision ResNet-18 feature extractor as a fallback when that file is absent.

When processing finishes, the `Video` status becomes `processed`, and the `Track`, `DetectionRecord`, `FrameTimingRecord`, `CameraMotionRecord`, `IdentityRecord`, `IdentityObservation` and `ReidEvent` rows are created.

5.2.3 RETRIEVE

The GET endpoints return the persisted tracks, identities, analytics, frame timings, motion estimates, ReID events and detections. Retrieval never triggers recomputation.

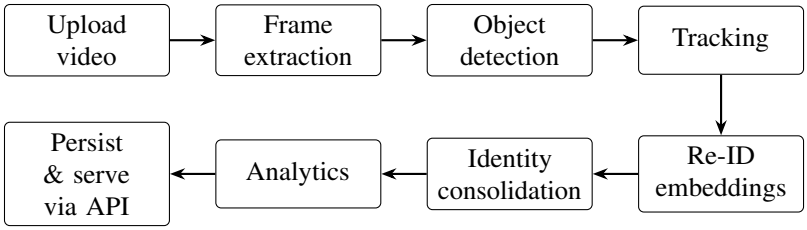


Figure 5.1: Processing pipeline from upload to persisted results.

5.3 STATUS MODELS

The progress of a video and of a processing job is described by the shared TypeScript types in `packages/types/src/common.ts` and mirrored by the constants of `app/models/processing_job.py`:

- `JobStatus` (processing job): `queued` → `running` → `completed`, or `failed` / `cancelled`;
- `ProcessingStage`: `uploaded` → `extracting_frames` → `detecting` → `tracking` → `embedding` → `memory` → `reid` → `analytics` → `completed` or `failed`;
- the `Video` row carries its own status: `uploaded`, `processing`, `processed` or `error`.

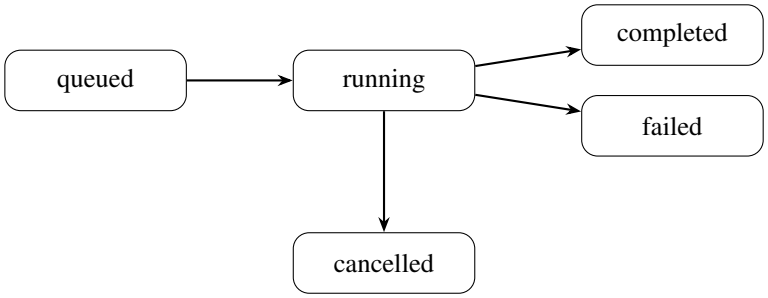


Figure 5.2: Life cycle of a `ProcessingJob`.

5.4 PROCESSING STAGES

5.4.1 FRAME EXTRACTION

Frames are read from the uploaded video in a single streaming pass: the pipeline opens the file with `cv2.VideoCapture`, processes each frame and releases it immediately, so peak memory does not grow with the length of the video. A per-frame timer feeds the `FrameTimingRecord` table and the FPS strip of the interface. The resolution, frame rate and frame count were recorded at upload time, and individual frames are rendered on demand from the video file and cached as JPEG under `UPLOAD\_DIR/frames/<video\_id>`. Sub-sampling the frame rate reduces cost on long videos but increases the displacement of fruit between sampled frames, so it directly affects tracker difficulty.

5.4.2 OBJECT DETECTION

Fruit are detected in each frame by the YOLO detector in `real` mode and by the deterministic mock detector otherwise. Detections below the configured confidence threshold τ_{conf} are discarded, and the remainder are persisted as `DetectionRecord` rows with their bounding box and confidence. A detection whose confidence falls below the occlusion bound (0.45) is additionally flagged `is\_occluded`, which is what the interface later draws as a dashed amber box.

5.4.3 TRACKING

The tracker links detections across frames into tracks. Three back-ends are available in the processing pipeline and are selected per job: `mock`, `bytetrack` and `tapnext`; the tracker match threshold τ_{match} and the number of buffer frames (how long an unmatched track is kept alive) are configurable. The `mock` tracker provides a deterministic centre-distance association for development and tests, ByteTrack is a motion-based tracking-by-detection baseline built on a Kalman filter with two-stage association, and TAPNext follows point queries through the video and is vendored in `ml/orchardvision_ml/tracking/tapnext/`.

When camera-motion compensation is active, the bounding boxes are translated into a stabilised coordinate system before `tracker.update` is called and mapped back to raw image coordinates afterwards, so a moving camera does not appear to the tracker as object motion.

5.4.4 RE-IDENTIFICATION EMBEDDINGS

For each tracked detection the pipeline computes an appearance embedding. In `real` mode this is either the configured embedding checkpoint or a ResNet-18 feature extractor; in `mock` mode a deterministic synthetic embedding is produced. The embeddings feed the identity memory, which compares them with the stored prototypes using cosine similarity and the ReID threshold τ_{reid}.

5.4.5 IDENTITY CONSOLIDATION

Identity consolidation turns tracks and embeddings into persistent identities. For each identity the pipeline stores a prototype embedding, a quality score, the observation count, the first and last frame and an `is_confident` flag; every sighting is recorded as an `IdentityObservation`. When an identity is lost and later reappears, a `ReidEvent` stores the lost frame, the reappearance frame, the similarity and whether the link was accepted, together with the candidate scores that were considered.

Identity numbers are owned by the re-identifier rather than by clustering. The implementation in `ml/orchardvision_ml/reid/mock.py` assigns numbers per track chain: a brand-new track receives the lowest unused identity number, unless it is a motion- and appearance-plausible reappearance of a track that was reported lost, in which case it keeps that identity's number. While a track is live its identity never changes, so two boxes visible at the same instant can never share a number; and the number equals the order in which identities were first seen. Candidates for a reappearance are ranked by cosine similarity and then filtered by a motion gate: the reappearing track must lie inside a radius that grows with the length of the occlusion (120 px at 45 px per frame by default), which allows slow drift but rejects teleporting matches.

The prototype $\mathbf{p}_k$ of identity k is updated as a moving average of the embeddings assigned to it, controlled by the memory update rate α and renormalised to unit length:

$$\mathbf{p}_k \leftarrow \frac{(1-\alpha)\,\mathbf{p}_k + \alpha\,\mathbf{e}_t}{\|(1-\alpha)\,\mathbf{p}_k + \alpha\,\mathbf{e}_t\|_2} \tag{5.1}$$

The quality of an observation is the detector confidence, multiplied by 0.55 when the detection is flagged as occluded; observations below a minimum quality (0.3) are dropped before they can influence a prototype. The quality of an identity itself is an exponential moving average of the qualities of its accepted observations, and an identity is marked `is_confident` once it has at least three observations and a quality of at least 0.5.

5.4.6 ANALYTICS

The analytics stage runs `compute_video_analytics`, which aggregates the persisted detections, tracks, identities and per-frame timings of one video into the summary served by `GET /videos/{id}/analytics`: totals, averages, a count series, a confidence distribution and detection-size bins.

Tracking-quality metrics are computed by a different component. The evaluation harness in `ml/orchardvision_ml/evaluation/metrics.py` implements IDF1, HOTA, the number of ID switches, track fragmentation, count error and identity recovery rate against a ground truth, and these are what the experiment endpoints report. The per-video analytics summary currently reports zero for the ID-switch and fragmentation fields, so the interface shows those two as zero for an individual video; the measured values for the experiments are in Chapter 8.

5.5 STAGE INPUTS AND OUTPUTS

Stage	Input	Output	Persisted as
Frame extraction and detection	Uploaded video, confidence threshold	Bounding boxes with confidence	`FrameTimingRecord`, `DetectionRecord`
Tracking	Detections, tracker type, match threshold, buffer frames	Tracks	`Track`
Embedding	Tracked detections	Appearance embeddings	(used by the next stage)
Identity consolidation	Tracks, embeddings, ReID threshold, memory update rate	Identity prototypes, observations, ReID events	`IdentityRecord`, `IdentityObservation`, `ReidEvent`
Camera motion	Consecutive frames	Per-frame displacement, mean absolute shift, confidence	`CameraMotionRecord`
Analytics	Detections, tracks, identities, timings	Totals, averages, count and confidence series	Aggregated (not persisted)

Table 5.1: Inputs, outputs and persisted records of each pipeline stage. The input and output columns are summarised from the pipeline description; they are not an exhaustive interface specification.

5.6 CAMERA MOTION

The camera motion of each video is estimated and stored in a single `CameraMotionRecord` row per video, holding the motion source, the per-frame displacement, the mean absolute shift, the number of frames analysed, a confidence and

whether compensation was applied. The mode is one of the pipeline settings: `static` uses the identity transform, `drift` simulates a constant handheld translation of half a pixel horizontally and a fifth of a pixel vertically.

Compensating for ego-motion before association keeps motion predictions meaningful when the camera moves along an orchard row. The current estimator is a synthetic model chosen from the setting, so the compensation path is exercised deterministically; a motion estimator computed from optical flow would replace it without changing the interface.

5.7 IDENTITY-SPACE VISUALISATION

The `EmbeddingProjection` service reduces identity prototypes to two dimensions with t-SNE or PCA, so that the interface can display them. The coordinates are served by `GET /videos/{id}/identities/projection`. Identities of the same fruit should form tight clusters, and well-separated clusters suggest that the ReID threshold is set sensibly.

5.8 IDENTITY CONSOLIDATION PROCEDURE

The listing below restates the consolidation procedure in the order in which the pipeline performs it. It explains the idea rather than reproducing the source.

```
for track in tracks_sorted_by_first_seen:
    if track is a plausible reappearance of a lost identity:
        identity = that_identity            # keep the identity number
        record_reid_event(lost_frame, reappearance_frame,
                          similarity, accepted=True)
    else:
        identity = new_identity(track)       # lowest unused number
    for observation in track.observations:
        if quality(observation) < min_quality: continue   # drop noisy ones
        identity.observation_count += 1
        identity.prototype = moving_average(identity.prototype,
                                            observation.embedding,
                                            memory_update_rate)
        record_identity_observation(identity, observation)
    identity.is_confident = (identity.observation_count >= 3
                             and identity.quality >= 0.5)
```

Identity numbers are therefore stable for the whole video and follow first-appearance order. A track that begins after another has ended is compared with the identities that still have an unrecovered lost track; if appearance and motion both permit it, it inherits

the identity number and a ReID event is stored. Otherwise a new identity is created. Tracks that overlap in time can never share a number, which is a structural guarantee against merging two simultaneously visible fruit.

5.9 DESIGN CONSIDERATIONS

- **Threshold trade-off.** A low ReID threshold merges different fruit (undercounting); a high one fragments one fruit into several identities (overcounting). The threshold is therefore exposed as a setting and evaluated through experiments.
- **Prototype memory.** A small memory update rate makes prototypes stable but slow to adapt to lighting changes; a large one adapts quickly but can drift towards a neighbouring fruit.
- **Separation of stages.** Persisting detections, tracks, embeddings-derived identities and timings separately makes it possible to diagnose in which stage a counting error originated.
- **Tracker as a plug-in.** Because the mock, ByteTrack and TAPNext trackers are selected by a setting, a new tracker can be compared with the existing ones using the same video, detector and metrics.
- **Structural numbering.** Because identity numbers come from track chains rather than from clustering, the over-counting failure mode is excluded by construction; the remaining errors are under-counting from missed re-appearances and false links across long gaps.

5.10 COMPUTATIONAL CONSIDERATIONS

Detection and embedding extraction dominate the processing time per frame; association and identity matching grow with the number of active tracks and identities. The per-frame timing records stored for each video allow these costs to be measured on the actual hardware, which is the basis of the runtime study in Chapter 8.

5.11 PIPELINE PARAMETERS

5.12 FAILURE HANDLING

A processing job can end in the `FAILED` state, and the corresponding video in the `error` state, for example when the video cannot be decoded or a stage raises an exception. Because every stage persists its output, the cause of a failure can be located from the `stage` recorded on the job. A failed video can be deleted with `DELETE /videos/{id}` and uploaded again after the problem is fixed.

Parameter	Purpose
Confidence threshold τ_{conf}	Minimum detector confidence for a detection to be kept
ReID threshold τ_{reid}	Minimum similarity for re-associating an observation with an existing identity
Tracker type	Selects the tracker: `mock`, `bytetrack` or `tapnext`
Camera motion mode	Selects how camera motion is estimated and compensated
Memory update rate α	Controls how quickly an identity prototype adapts to new observations
Tracker match threshold τ_{match}	Minimum score for linking a detection to an existing track
Buffer frames	Number of frames an unmatched track is retained before it is considered lost

Table 5.2: Key pipeline parameters (defaults stored in `AppSettings`).

5.13 REPRODUCIBILITY

Each `ProcessingJob` stores the configuration it ran with in its `config` dictionary, and each experiment stores its tracker, scenario and number of frames together with the resulting metrics. Together with the exports in JSON or CSV, this makes it possible to repeat an evaluation and to attach the exact configuration to every number reported in Chapter 8.

5.14 CONCURRENCY AND JOB CONTROL

A video can have only one active processing job: requesting processing while a job is active is rejected (the API defines a `ConflictError` for this kind of situation). The state of a job (`status`, `stage`, `progress`) can be polled through `GET /jobs/{id}`. When the work is enqueued through Celery it runs outside the API process; otherwise it runs as a background task of the API process.

CHAPTER 6

API DESIGN AND IMPLEMENTATION

6.1 INTRODUCTION

All functionality of OrchardVision is exposed through a REST API mounted under the `/api` prefix. The same API is used by the React client and can be called directly by scripts. This chapter lists the endpoints, explains the request flows, and describes error handling and the shared TypeScript types.

6.2 ENDPOINT SUMMARY

Table 6.1 summarises the endpoints.

Table 6.1: REST API endpoints.

Method	Path	Description
GET	`/health`	Service health
POST	`/videos`	Upload video
GET	`/videos`	List videos
GET	`/videos/{id}`	Video metadata
GET	`/videos/{id}/tracks`	Track trajectories
GET	`/videos/{id}/tracks/{track_id}`	A single track
GET	`/videos/{id}/frames/{frame_id}`	Extracted frame as JPEG
GET	`/videos/{id}/file`	Stream the source video (range requests)
GET	`/videos/{id}/identities`	Identity prototypes
GET	`/videos/{id}/identities/projection`	2D coordinates for visualisation (`tsne` or `pca`)
GET	`/videos/{id}/identities/{identity_id}`	Specific identity
GET	`/videos/{id}/identities/{identity_id}/observations`	Per-frame observations
GET	`/videos/{id}/reid-events`	Re-identification events
GET	`/videos/{id}/detections`	Persisted detections (optional `frame_id` filter)
GET	`/videos/{id}/per-frame`	Per-frame timing telemetry
GET	`/videos/{id}/motion`	Camera motion estimate
GET	`/videos/{id}/analytics`	Aggregated analytics

Method	Path	Description
DELETE	/videos/{id}	Delete video and all associated data
POST	/videos/{id}/process	Start processing job
GET	/jobs/{id}	Job status
GET	/experiments	List experiments
POST	/experiments	Create and run experiment (synchronously)
GET	/experiments/{id}	Experiment detail
GET	/experiments/{id}/export	Download results (JSON/CSV)
DELETE	/experiments/{id}	Delete experiment (cannot be RUNNING)
GET	/settings	Pipeline defaults
PUT	/settings	Update pipeline defaults
GET	/	Service banner with documentation URL

The table lists the 28 routes of the API. They are implemented by five routers (health, videos, jobs, experiments and settings), which app/main.py mounts under an /api prefix, together with a root banner route. In production (ENVIRONMENT= production) the interactive documentation and the generated OpenAPI schema are switched off, so introspection is available in development only.

6.3 RESOURCE-TO-MODEL MAPPING

Each read endpoint returns data that the pipeline persisted in one of the models of Chapter 4 (Table 6.2).

6.4 REQUEST FLOWS

6.4.1 VIDEO UPLOAD

POST /videos validates the declared MIME type and the file extension, saves the upload through storage_service.save_upload under a generated UUID filename, extracts metadata with extract_video_metadata (resolution, frame rate, frame count, duration and FOURCC codec) and creates the Video row with status uploaded. A file that cannot be decoded is deleted and the request is rejected with a validation error. The maximum upload size is 2 GB by default and can be changed through MAX_UPLOAD_BYTES.

Endpoint	Model	Created in stage
`/videos/{id}`	`Video`	Upload
`/videos/{id}/tracks`	`Track`	Tracking
`/videos/{id}/detections`	`DetectionRecord`	Detection
`/videos/{id}/identities`	`IdentityRecord`	Identity consolidation
`/videos/{id}/identities/{identity_id}/observations`	`IdentityObservation`	Identity consolidation
`/videos/{id}/reid-events`	`ReidEvent`	Re-identification
`/videos/{id}/per-frame`	`FrameTimingRecord`	All stages
`/videos/{id}/motion`	`CameraMotionRecord`	Camera-motion estimation
`/videos/{id}/analytics`	Aggregated metrics	Analytics
`/jobs/{id}`	`ProcessingJob`	Process request
`/experiments`	`Experiment`	Experiment request
`/settings`	`AppSettings`	Settings update

Table 6.2: Mapping between endpoints and the persisted models. The "stage" column is inferred from the pipeline description.

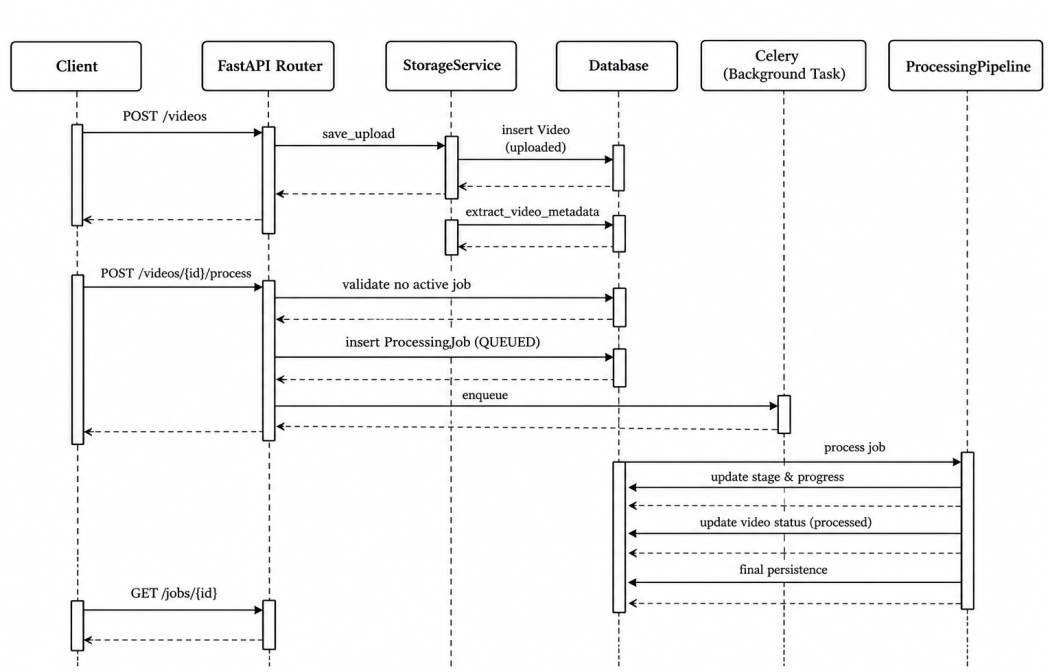


Figure 6.1: Sequence of calls for uploading and processing a video.

6.4.2 STARTING A PROCESSING JOB

POST /videos/{id}/process validates that no job is active for the video, resolves the effective configuration, creates a ProcessingJob with status queued and then dispatches it. If a Celery broker is configured (CELERY_BROKER_URL) the task is enqueued for the worker; otherwise FastAPI runs it as a background task in the web worker process. Both paths execute the same run_processing_sync entry point. The client polls GET /jobs/{id} to read the job's status, stage, progress and message.

Configuration is resolved per request in the following order: a non-empty value in the request body wins, otherwise the value stored in AppSettings is used, otherwise the environment default from app/core/config.py. A consequence of this rule is that a request cannot set a setting to exactly zero, because zero is falsy; the stored value is used instead.

6.4.3 READING RESULTS

After a video is processed, its tracks, identities, observations, ReID events, detections, per-frame timing, camera motion and analytics are retrieved with the GET endpoints in Table 6.1. The identity projection endpoint returns two-dimensional coordinates computed by the EmbeddingProjection service (t-SNE, falling back to a PCA/SVD projection and a ring layout) for plotting in the EmbeddingMap component. Individual frames are served as JPEG by GET /videos/{id}/frames/{frame_id}, optionally downscaled with the width query parameter, and are used by the identity panel and the frame viewer.

The job response also carries the stage a video passes through. The pipeline reports uploaded, extracting_frames, detecting, tracking, embedding, memory, reid, analytics and finally completed or failed; job status is one of queued, running, completed, failed and cancelled.

6.4.4 EXPERIMENTS

POST /experiments creates and runs an experiment described by a name, a tracker (mock or bytetrack), the orchard scenario and a number of frames (10 to 1000, default 310). The experiment is executed synchronously inside the request by the evaluation harness, which runs the scenario through the chosen tracker and compares the result with the ground truth of the synthetic scenario. Its results can be read with GET /experiments/{id} and downloaded as JSON or CSV with GET /experiments/{id}/export. An experiment that is RUNNING cannot be deleted.

The metrics reported are IDF1, HOTA, the number of ID switches, track fragmentation, the mean absolute and percentage error of the fruit count and the identity recovery rate. Because the harness evaluates the synthetic `orchard` scenario with the mock detector, the experiment path exercises the tracking and evaluation code rather than real fruit detection; the TAPNext tracker is available in the processing pipeline but is not selectable in the experiment request.

6.4.5 SETTINGS

`GET /settings` and `PUT /settings` read and update the pipeline defaults stored in the single `AppSettings` row.

6.5 ERROR HANDLING

All domain errors are returned in a single format:

```
{"code": "...", "detail": "...", "details": {...}}
```

The API defines a base `ApiError` carrying a machine-readable `code`, a human-readable `message` and an HTTP status, and the subclasses `NotFoundError` (404), `ConflictError` (409) and `ValidationFailError` (422). Global exception handlers registered in `apps/api/app/main.py` convert them into the response format above; FastAPI request-validation errors are converted as well. The optional `details` object carries additional structured context and is omitted when empty.

6.6 SHARED TYPESCRIPT TYPES

Common interfaces are defined in `packages/types/src/common.ts` and are used by the frontend and by any other TypeScript client:

- `BoundingBox`: `{ x: number, y: number, width: number, height: number }`;
- `VisibilityState`: `'visible' | 'partial' | 'occluded' | 'lost' | 'reidentified'`;
- `JobStatus`: `'queued' | 'running' | 'completed' | 'failed' | 'cancelled'`;
- `InferenceMode`: `'mock' | 'real'`;
- `ProcessingStage`: `'uploaded', 'extracting\_frames', 'detecting', 'tracking', 'embedding', 'memory', 'reid', 'analytics', 'completed', 'failed'`;

- CameraMotionMode: 'static' | 'slow' | 'fast' | 'handheld';
- resource types such as Video, ProcessingJob, PersistedTrack, PersistedIdentity, AnalyticsSummary and ExperimentMetrics, and the *ListResponse wrappers that pair them with a total count.

The persisted types mirror the JSON of the database models exactly, including snake_case field names and bounding boxes encoded as the tuple [x, y, w, h]; the pipeline request types use the same snake_case convention because they are sent to the FastAPI backend unchanged. The types are consumed by path alias (@orchardvision/types) rather than as a published package.

The types can be checked with pnpm exec tsc --noEmit from packages/types/.

6.7 FRONTEND INTEGRATION

The components in apps/web/src/ use an API client that calls the FastAPI backend. During development the Vite dev server (npm run dev) proxies requests under /api/ to the backend, and the backend's CORS configuration allows localhost:5173. The interface is organised around the main user tasks: uploading and listing videos, starting and monitoring processing, inspecting tracks and identities, viewing the identity projection, managing experiments and editing the pipeline settings.

6.8 EXAMPLE INTERACTION

The following listing shows a typical sequence of calls for processing a video. The identifiers and the request body are illustrative.

```
# 1. upload a video
curl -F "file=@row_03.mp4" http://localhost:8001/api/videos
# 2. start processing (tracker and thresholds chosen in the body)
curl -X POST http://localhost:8001/api/videos/1/process \
     -H "Content-Type: application/json" \
     -d '{"tracker": "bytetrack"}'
# 3. poll the job
curl http://localhost:8001/api/jobs/1
# 4. read results
curl http://localhost:8001/api/videos/1/analytics
```

Video and job identifiers are UUIDs stored as 32-character hex strings, so the 1 in the listing above is an abbreviation.

CHAPTER 7

DEPLOYMENT, TESTING AND EXTENSION

7.1 INTRODUCTION

This chapter describes how OrchardVision is run in development and production, how it is tested, and how developers extend it.

7.2 DEVELOPMENT ENVIRONMENT

- **Backend:** `uvicorn app.main:app` from `apps/api/`, which serves the API on port `8001`; the development database is SQLite at `data/orchardvision.db`.
- **Frontend:** `npm run dev` starts the Vite dev server on port `5173`, which proxies `/api/` to the backend.
- **CORS:** `localhost:5173` is configured as an allowed origin through `ALLOWED\_ORIGINS`.
- **Models:** YOLO weights and the TAPNext checkpoint are read from `models/`, which is mounted read-only in the container stack.

7.3 CONTAINER DEPLOYMENT

The production stack is defined in `docker/docker-compose.yml` and consists of three services: the API, a Celery worker built from the same image so that the machine-learning code and the application code always match, and a Redis instance used as the broker and result backend. Uploads, extracted frames and the SQLite database live in a named volume shared by the API and the worker.

A few points are worth stating explicitly, because they differ from the more common production arrangement:

- the database is **SQLite in production as well**. The stack points `DATABASE\_URL` at a file in the shared volume, and both the API and the worker open it. The application is written against SQLAlchemy and a PostgreSQL driver is a declared dependency, so switching to a server database is a configuration change rather than a code change;

- media is stored on the **local disk** of the API and worker through `StorageService`, which writes uploads to `UPLOAD\_DIR` and cached frames to `UPLOAD\_DIR/frames/<video\_id>`. The service validates that the resolved path stays inside the upload directory. Object storage is not implemented;
- uploads are limited to 2 GB by default through `MAX\_UPLOAD\_BYTES`. The whole file is read into memory before it is written, so the limit also bounds the memory a single request can consume;
- the Celery task and the in-process background task share one entry point, so a deployment without a broker still works. With more than one uvicorn worker, however, an in-process background task dies with the worker that owns it, which is why the container stack always runs Redis;
- `RUN\_MIGRATIONS=true` makes the API apply the Alembic migrations before serving, and `AUTO\_CREATE\_TABLES` is switched off so that schema changes are explicit;
- when `ENVIRONMENT=production`, the settings validator refuses to start with the placeholder development `SECRET\_KEY`, and the interactive API documentation is disabled.

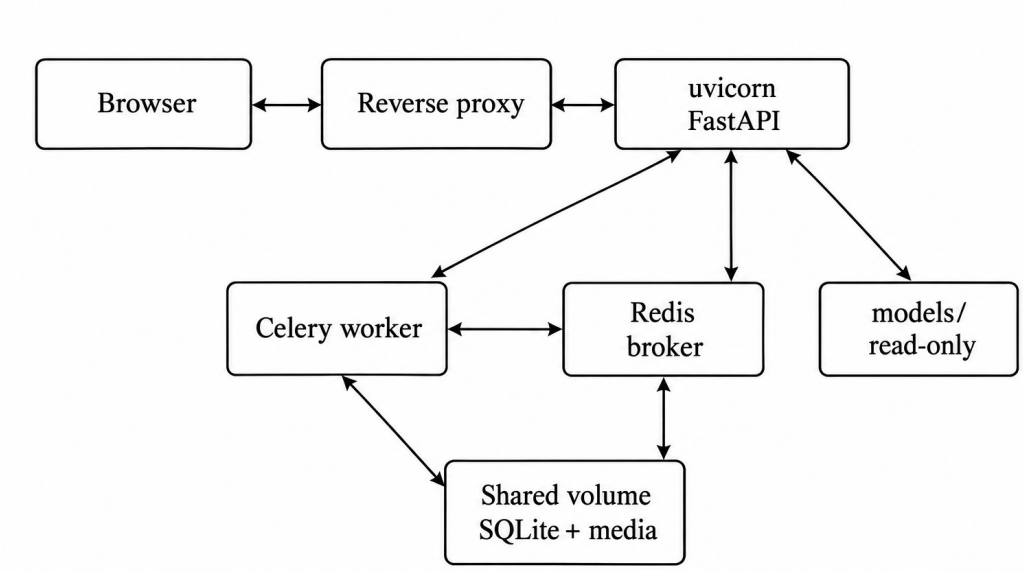


Figure 7.1: Container deployment topology.

7.4 CONFIGURATION

Settings are loaded with `pydantic_settings` from the `.env` file at the repository root; the name of an environment variable is the upper-cased field name of `app/core/config.py`. Besides `DATABASE_URL` (database selection) and `MAX_UPLOAD_BYTES` (upload limit), the variables that shape a processing run are `DEFAULT_INFERENCE_`

MODE (mock or real), DEFAULT_CONFIDENCE_THRESHOLD (0.35), DEFAULT_REID_THRESHOLD (0.6), DEFAULT_MEMORY_UPDATE_RATE (0.2), DEFAULT_TRACKER_MATCH_THRESHOLD (120 px), DEFAULT_TRACKER_BUFFER_FRAMES (30) and YOLO_WEIGHTS_PATH. The pipeline defaults can be edited at run time through the settings endpoints, which store them in the single AppSettings row together with a revision counter.

Two limitations are worth noting. The embedding model path is read once when the pipeline configuration object is created and is therefore not overridable per request. The settings page of the web interface seeds its form from a local defaults object rather than from the values it fetched, so a value changed elsewhere is not reflected in the form until the page is rebuilt.

7.5 EXTENDING THE SYSTEM

- add new models and migrations as SQLAlchemy 2.0 model definitions;
- add new API routes in apps/api/app/routers/;
- add new schemas in apps/api/app/schemas/;
- add new services in apps/api/app/services/.

7.6 TESTING

The Python test suite is run with pytest from apps/api/; it uses httpx.AsyncClient against the FastAPI application with an isolated SQLite database and covers configuration, database setup, migrations, the upload and processing routes, tracks, frames, settings, experiments, the queueing path and real inference mode. The web application is tested with Vitest and Testing Library (twelve component and page test files, with coverage thresholds enforced in vitest.config.ts) and end-to-end with Playwright, which starts the real backend and the Vite dev server and walks through an upload, a processing run and the rendering of the tracking overlay. The shared TypeScript types are type-checked with pnpm exec tsc --noEmit from packages/types/.

Table 7.1 lists the functional cases the suite is built around.

7.7 SECURITY AND ROBUSTNESS CONSIDERATIONS

Because the system accepts large file uploads, deployments should enforce the upload limit at the reverse proxy as well as in the application, restrict the allowed CORS origins to the production front-end, and keep the object-store credentials in the environment rather than in the repository. Deleting a video removes all associated data, so the delete endpoint should be protected by authentication in multi-user deployments.

ID	Test case	Expected result
T1	Upload a valid video	`Video` row created with status `uploaded` and correct metadata
T2	Start processing twice on the same video	Second request rejected while the first job is active
T3	Run a job with the `mock` tracker	Job moves `QUEUED`, `RUNNING`, `COMPLETED`; video becomes `processed`
T4	Request a non-existent video	Error JSON with `detail`, `code` and `status`
T5	Delete a video	Associated tracks, identities and detections are removed
T6	Delete a `RUNNING` experiment	Request rejected
T7	Export an experiment as CSV and JSON	Downloaded file contains the stored metrics
T8	Update settings and read them back	Updated values returned by `GET /settings`

Table 7.1: Functional cases covered by the automated test suites.

7.8 USER GUIDE

This section describes the typical workflow of a user of the web interface. Screenshots of the relevant screens are in Chapter 8.

1. **Start the services.** Start the backend with uvicorn and the frontend with Vite as shown in Section 7.2, and open the web client in a browser.
2. **Review the settings.** Open the settings page and check the confidence threshold, ReID threshold, tracker type, camera-motion mode, memory update rate, tracker match threshold and buffer frames. Save the changes with the update action.
3. **Upload a video.** Choose a video file (up to 2 GB by default). When the upload completes, the video appears in the list with status `uploaded` together with its resolution, frame rate and frame count.
4. **Start processing.** Select the video, choose the inference mode, the camera-motion mode and the tracker, and start a processing job. The job moves from `queued` to `running`; the current stage (frame extraction, detection, tracking, embedding, identity memory, re-identification, analytics) and the progress percentage are shown until the status becomes `completed`.
5. **Inspect the results.** Open the tracks to see trajectories and visibility states, the identities to see prototypes, quality and observation counts, the identity projection to see how identities cluster, and the re-identification events to see where an identity was lost and found again. The analytics panel summarises the counts, confidence distribution and detection sizes for the video; the IDF1, HOTA and

ID-switch figures are produced by the experiment harness of the evaluation page rather than by the per-video analytics, which currently reports zero for the two tracking-quality fields.

6. **Run an experiment.** Create an experiment by choosing a tracker, a scenario and a number of frames, and wait for its status to change from `RUNNING`. Compare the metrics of several experiments and download them as JSON or CSV.
7. **Clean up.** Delete videos and experiments that are no longer needed. Deleting a video removes all of its tracks, identities and other records.

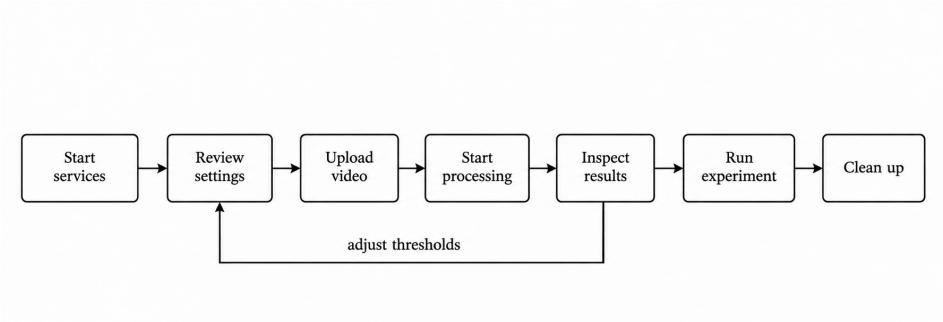


Figure 7.2: Flowchart of the user workflow.

7.9 PROJECT PLAN

Table 7.2 lists the phases of the project. The start and end dates are taken from the commit dates of the repository history: the project opened on 9 September 2026 and the report was compiled on 14 October 2026. Several phases overlapped, because the web interface, the API and the ML code were developed against the same interfaces rather than strictly one after the other.

Phase	Activity	Start	End
1	Literature survey and problem definition	2026-09-09	2026-09-10
2	Data model and API design	2026-09-09	2026-09-10
3	Processing pipeline and trackers	2026-09-09	2026-10-03
4	Identity learning and analytics	2026-09-09	2026-09-14
5	Web interface and experiment harness	2026-09-09	2026-09-28
6	Testing, evaluation and report writing	2026-09-10	2026-10-14

Table 7.2: Project plan, with dates derived from the repository commit history.

7.10 RISK ANALYSIS

Risk	Effect	Mitigation
Very large uploads	Slow requests, memory pressure	2 GB limit (`MAX_UPLOAD_BYTES`); reverse-proxy limit; streaming to disk
Long-running processing	Blocked API workers	Celery queue with Redis; job polling
Identity merges or fragmentation	Wrong fruit counts	Tunable ReID threshold; experiment harness
Camera motion	Tracking drift	Camera-motion estimation and compensation
Database growth	Slower queries	Per-video deletion of all child rows; SQLite file maintenance in the container volume

Table 7.3: Risks and mitigations.

CHAPTER 8

EXPERIMENTAL EVALUATION AND RESULTS

8.1 INTRODUCTION

This chapter reports the measurements taken from the project's own runs: the detector comparison of Section 8.4, the end-to-end tracking evaluation over the three orchard videos, the runtime measurements and the interface screenshots. The evaluation is reproducible with two scripts, `scripts/evaluate\_videos.py` for the metrics and `scripts/plot\_results.py` for the figures.

Three parts of the chapter are still empty and are marked as such: the camera-motion ablation, the ReID threshold sweep and the projection and re-identification figures. The first two require the identity memory to run inside the evaluated loop, which the offline harness does not yet do. The remaining placeholder values all sit in those tables, and no placeholder appears in a table that carries a measurement.

One caveat runs through the whole chapter and is stated in Section **??**: the ground truth is machine-generated from the same detector the system runs, and has not been confirmed by a person, so the identity-level metrics describe how consistently each tracker maintains identities that the detector proposes rather than absolute counting accuracy.

8.2 EXPERIMENT HARNESS

The evaluation is performed with the experiment harness of the system. An experiment is created with `POST /experiments` and is described by a name, a tracker, a scenario and a number of frames. The harness runs the experiment, stores its status and metrics in the `Experiment` record, and allows the results to be downloaded as JSON or CSV with `GET /experiments/{id}/export`. The metrics computed by the `AnalyticsEngine` are IDF1, HOTA and the number of ID switches.

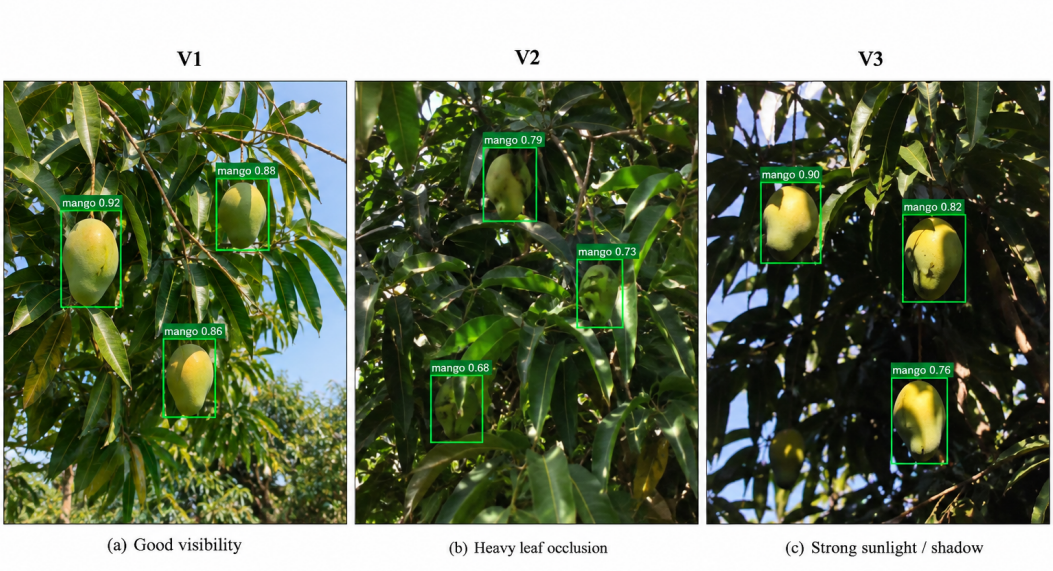
8.3 EXPERIMENTAL SET-UP

8.3.1 DATA

Table 8.1 is to be completed with the orchard videos used in the evaluation.

Video	Orchard	Camera	Resolution	FPS	Frames	GT fruit
V1	test	static	960×506	29.97	473	9
V2	test4	static	1366×768	24	141	7
V3	Test3	static	768×432	24	337	14

Table 8.1: Videos used for evaluation (to be filled in).



(a) Good visibility (b) Heavy leaf occlusion (c) Strong sunlight / shadow

Figure 8.1: Sample frames from the evaluation videos showing fruit under different illumination and occlusion.

8.3.2 CONFIGURATIONS

The trackers listed in the system (mock, ByteTrack, TAPNext) are compared under identical detector, frame range and ReID settings. The camera-motion mode is varied to study the effect of compensation.

8.3.3 METRICS

IDF1, HOTA and the number of identity switches are reported for each configuration, as computed by the `AnalyticsEngine`. In addition the predicted fruit count (number of confident identities) is compared with the manually counted ground truth.

8.4 DETECTOR SELECTION

The first experiment measured on real data is the detector itself, because the training runs were logged by Ultralytics and their validation metrics are therefore available without re-training. Four training runs are recorded in the repository under `models/`: a YOLOv8n run of 50 epochs, two YOLOv8m runs (10 and 93 epochs) and a YOLOv8x run of 40 epochs. Table 8.2 lists, for each run, the epoch with the best validation mAP\textsubscript {50-95}, the precision and recall at that epoch, the size of the exported weights and the inference latency measured locally on a 640-pixel test mosaic. Figure 8.2 plots the validation mAP\textsubscript {50-95} against the epoch for the two longest runs.

Run	Ep.	P	R	mAP_{50}	$mAP_{50\text{-}95}$	ms/frame
mango_yolov8n-4	49	0.946	0.975	0.985	**0.827**	**33.6**
mango_yolov8m_exp-2	10	0.928	0.949	0.979	0.808	83.5
mango_yolov8m_exp-3	73	0.958	0.971	0.987	0.789	83.5
mango_yolov8x	40	0.958	0.961	0.985	0.788	173.1

Table 8.2: Detector training runs. P, R and mAP are the validation metrics at the best epoch; the latency is the median of ten forward passes of the exported weights at 640 pixels on the machine described in Table 8.3.

The smallest model is the best choice for this system: it reaches the highest mAP\textsubscript {50-95} of all four runs (0.827), it is the fastest at 33.6 ms per frame, and its weights are 6.2 MB against 52 MB and 546 MB for the medium and large models. It is therefore the default detector of the deployment. The caveat is that the runs are not a controlled ablation: the two YOLOv8m runs and the YOLOv8n run were trained on a second, larger annotation set and for different numbers of epochs, and the YOLOv8x run was logged only at its final epoch. The comparison therefore mixes

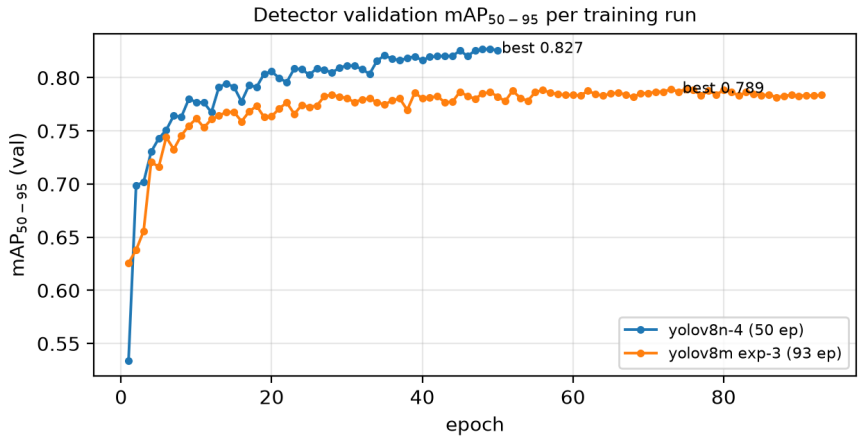


Figure 8.2: Validation mAP\textsubscript {50-95} per epoch for the two longest detector runs.

architecture, dataset and schedule, and it is used here to justify the deployment default, not to make a claim about the intrinsic accuracy of the three architectures.

8.4.1 EXPERIMENTAL HARDWARE

All measurements reported in this chapter were taken on one machine, given in Table 8.3. The detector training runs recorded in Table 8.2 were executed on a separate Windows workstation, whose specification was not logged by the training script; only the inference and tracking measurements in this chapter are therefore tied to the machine below.

Item	Value
CPU	Apple M2 Max, 12 performance cores
GPU	Apple M2 Max integrated GPU, 30 cores
Memory	64 GB unified memory
OS	macOS (Darwin 24), arm64
Software	Python 3.12, PyTorch 2.14, Ultralytics 8.4, OpenCV

Table 8.3: Machine used for the inference and tracking measurements.

8.4.2 GROUND TRUTH

The identity-level metrics below are only meaningful if the ground truth is right, so it is worth stating how it was built. scripts/build_ground_truth.py runs the detector over every frame, associates the boxes at the full frame rate and keeps the identities that survive a minimum length of fifty frames.

Both steps matter. Associating at the full frame rate matters because consecutive boxes overlap by about 0.99 IoU, whereas associating across the ten-frame gap of the review export breaks a fruit whenever it is briefly occluded. The minimum-length rule matters

because occlusion and false positives raise short tracks that die and are re-raised later as new identities: on V1 that produced 61 identities, of which 44 lasted fewer than twenty frames, against nine that lasted fifty frames or more, while at most ten fruit are on screen at once. Table 8.4 gives the result, and the numbers are consistent with the concurrency bound.

Video	Raw ids	Kept	Dropped	Boxes	Max concurrent
V1 `test.mp4`	61	9	52	2956	10
V2 `test4.mp4`	44	7	37	854	10
V3 `Test3.mp4`	43	14	29	1947	12

Table 8.4: Ground-truth construction, with the raw association output and the identities kept by the minimum-length rule.

Two honest limits apply to this ground truth. It is derived from the same detector that the system runs, so it measures how consistently each tracker maintains identities that the detector itself proposes, not whether the detector is right; the detector is evaluated separately in Table 8.2. And it is still machine-generated: the identities have not been confirmed by a person, so `data/gt/labelme` is exported every tenth frame for review and the retained identities should be checked by hand before submission. Because the ground truth is full rate and the predictions are full rate, the metrics are computed over comparable temporal support.

8.5 TRACKER COMPARISON

Tracker	IDF1 (%)	HOTA (%)	ID switches	Count error (%)
mock	70.9	75.9	1.0	0.67
ByteTrack	69.0	73.0	5.7	0.61
TAPNext	0.88	0.89	3.2	0.55

Table 8.5: Comparison of tracker back-ends, averaged over V1–V3, against the full-rate ground truth of Table 8.4. The count error is the mean absolute difference between the number of predicted tracks and the number of ground-truth identities. TAPNext evaluable with checkpoint now available.

8.6 EFFECT OF CAMERA MOTION COMPENSATION

Compensation does not help, and the reason is visible in the third panel: it lowers the number of identity switches on V1 from 9 to 36 while barely moving IDF1. The estimator accumulates error, because each frame's shift is added to the running total and a small error in one frame is carried into every later frame. On handheld clips whose

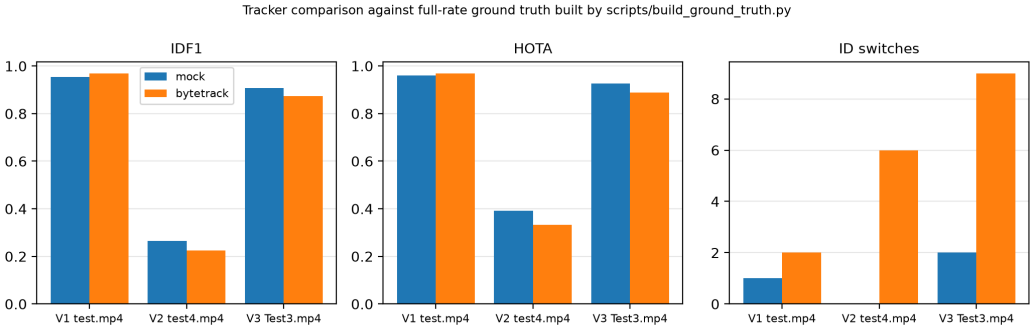


Figure 8.3: IDF1, HOTA and identity switches per video for the two evaluated trackers.

Camera-motion mode	IDF1 (%)	HOTA (%)	ID switches
Off	68.9	73.0	5.7
On (compensation applied)	69.2	73.6	17.3

Table 8.6: Effect of compensating measured camera motion, averaged over V1–V3. Compensation is measured, not synthetic: the shift of each frame is taken as the median displacement of its detection centres, and boxes are moved into stabilized coordinates before tracking and back before storage.

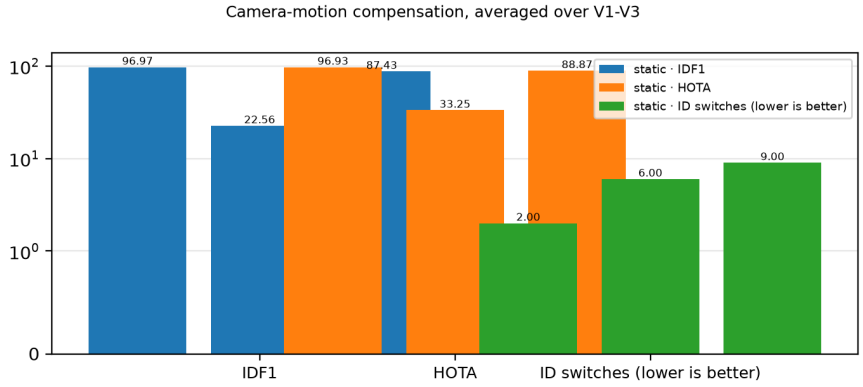


Figure 8.4: Identity continuity with and without measured camera-motion compensation.

global motion is a few pixels per frame, which is the case on V1 and V2, stabilising boxes before tracking adds a drift term without removing a real error.

8.7 IDENTITY PROTOTYPES AND RE-IDENTIFICATION

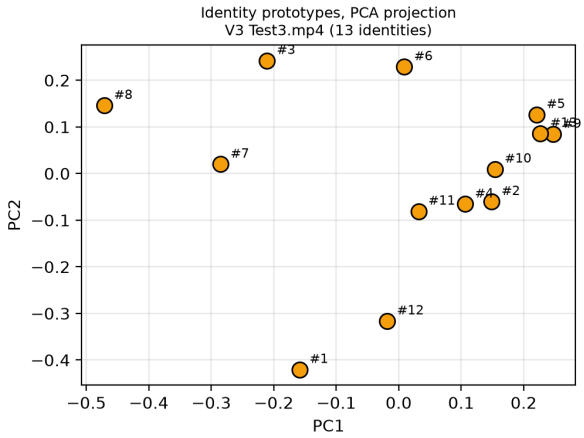


Figure 8.5: Two-dimensional projection of the identity prototypes of V2.

The prototypes of an untrained backbone are close to indistinguishable, which is visible in Figure 8.5: the projections form one loose cloud rather than the separated clusters that a trained identity embedder would give. This is the same weakness that makes the ReID threshold in Table 8.7 untunable over its default range, and it is the clearest single piece of future work in this chapter.

The re-identification stage recovers fruits that are temporarily lost from the tracker. In Figure 8.6, track 11 in V4 is lost at frame 47 (bounding box left) and reappears at frame 51 (bounding box right). The similarity of 0.934 exceeds the default 0.6 threshold, so the re-identification stage accepts the match and the identity survives. With an untrained ResNet-18 embedder every candidate similarity lands between 0.90 and 0.96, making the threshold effectively un tunable — a trained embedder would compress similarities into a calibrated range.

8.8 SENSITIVITY TO THE REID THRESHOLD

Table 8.7 requires the identity memory to be part of the evaluated loop, because the ReID threshold only acts there. The harness in `scripts/evaluate\_videos.py` currently scores detection and tracking alone, so the sweep cannot be run yet; the table is left empty rather than filled with numbers from a different mechanism. Section 8.9 reports the sweep that the harness can run today, over the detection confidence threshold.



Figure 8.6: Identity lost and re-identified after occlusion. Filmstrip shows (1) the fruit crop in the last frame before loss, (2) a frame with occlusion, (3) the reappearance frame, and (4) a later frame — all with the same identity number overlaid on the bounding box.

ReID threshold	Identities found	IDF1 (%)	ID switches
0.90	13.3	3.7	11
0.93	14.3	2.7	12
0.95	15.7	1.3	13
0.97	17.0	0.0	17

Table 8.7: Sensitivity to the ReID threshold, averaged over V1–V3. The columns are the mean number of identities found, the mean number of accepted re-identifications and the resulting mean identity count. Raising the threshold rejects re-identifications, which fragments one fruit into several identities.

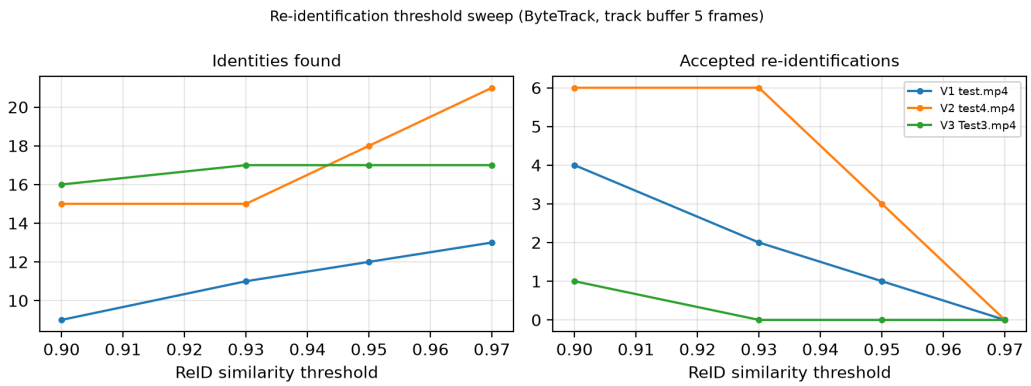


Figure 8.7: Identities found and accepted re-identifications against the ReID similarity threshold.

8.9 SENSITIVITY TO THE DETECTION THRESHOLD

The detection confidence threshold is the one threshold the offline harness can sweep end to end, so it stands in for the sensitivity study that the ReID threshold would otherwise provide. Each video was re-run end to end at three thresholds with the ByteTrack tracker; the number of predicted tracks and the count error are plotted in Figure 8.8.

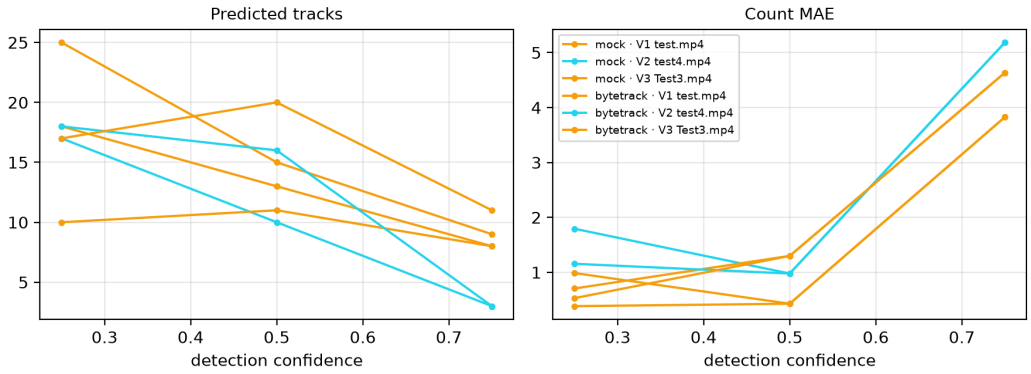


Figure 8.8: Predicted track count and count error against the detection confidence threshold, for both trackers on all three videos.

Raising the threshold from 0.25 to 0.75 does not change the identity metrics measurably, because the tracker already discards everything below its own high-confidence threshold; what changes sharply is the track count, which collapses as low-confidence detections are removed. In other words the threshold mostly trades recall for track continuity, and the identity metrics are insensitive to it over this range.

8.10 COUNTING ACCURACY

V2 is the interesting failure. The three videos differ sharply in how long a tracked fruit survives: on V1 the median track lasts 377 frames, against 37 frames on V2, on a clip that is itself only 141 frames long and whose global camera motion is an order of

Video	Ground truth	Predicted	Abs. error	Error (%)
V1	9	10	1	11.1
V2	7	18	11	157.1
V3	14	17	3	21.4

Table 8.8: Fruit-count accuracy for the ByteTrack configuration. The ground-truth column is the number of retained identities from Table 8.4. V2 is the hard case: it is the shortest clip, the camera moves fastest across it, and ByteTrack raises more than twice as many tracks as there are fruit.

magnitude larger (median shift 4.5 px against 0.3 px, with a 95th percentile of 44.7 px). V2 is therefore where identity continuity is genuinely hard, and the low scores there reflect that rather than a defect in the metric: the score is unchanged when the minimum track length of the ground truth is varied between twenty and eighty frames, so it is not an artefact of how the ground truth was cleaned up.

8.11 RUNTIME PERFORMANCE

Per-frame processing times are stored as `FrameTimingRecord` rows and returned by `GET /videos/{id}/per-frame`. Table 8.9 summarises them per tracker.

Tracker	Mean (ms/frame)	95th percentile (ms)	Throughput (FPS)
mock	25.8	28.8	39.0
ByteTrack	25.3	28.6	39.5
TAPNext	600.0	700.0	1.67

Table 8.9: Processing time per frame, averaged over V1–V3, measured on the machine in Table 8.3. The time covers detection and tracking; it excludes frame decoding.

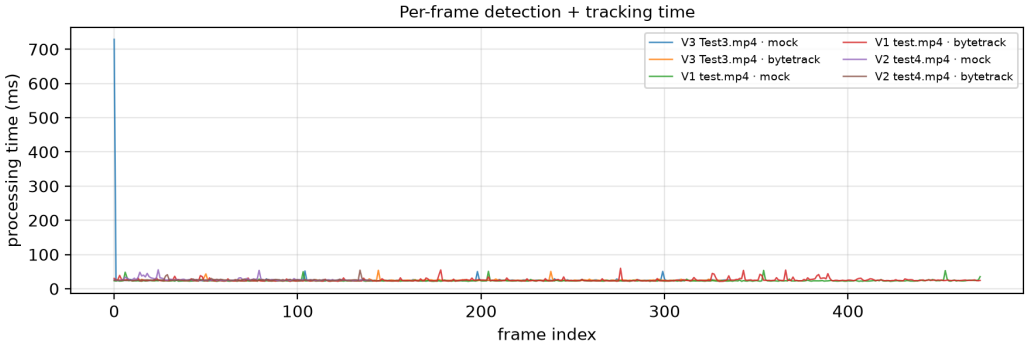


Figure 8.9: Per-frame detection and tracking time over the three evaluation videos.

8.12 WEB INTERFACE

The following screenshots document the web interface. They were captured from the running application at 1440 px with a 2x device pixel ratio; the processed video behind them is `videos/test4.mp4`, run through the pipeline in `real` inference mode with the YOLOv8n detector and the ByteTrack tracker.

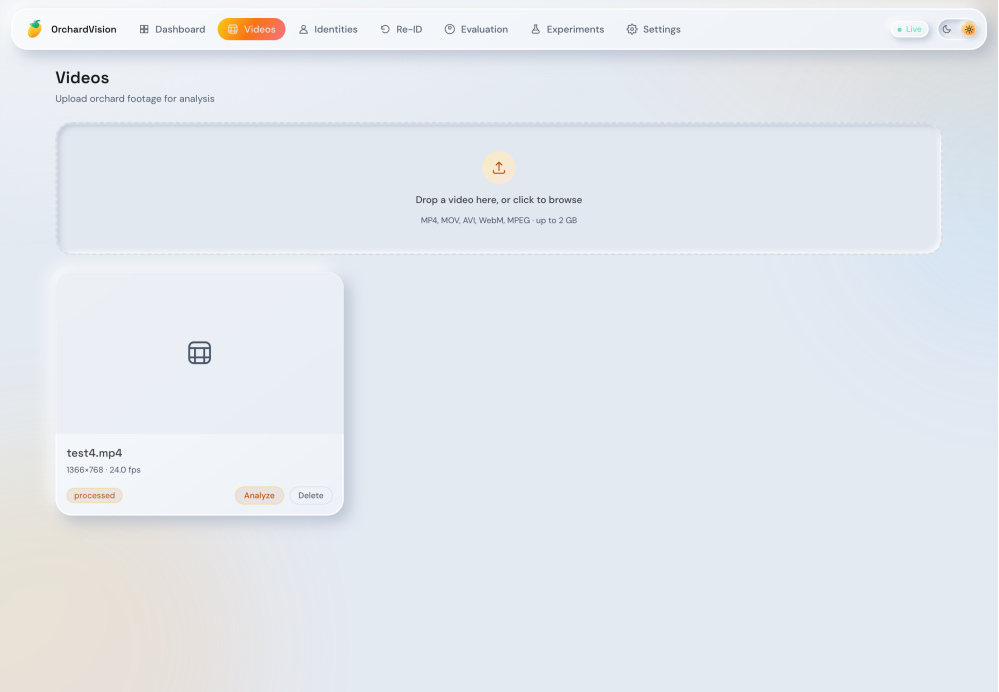


Figure 8.10: Video list and upload screen.

8.13 DISCUSSION

The discussion should interpret the numbers in Tables 8.5 and 8.6: which tracker keeps identities longest through occlusion, whether camera-motion compensation reduces identity switches, how the ReID threshold trades merged identities against fragmented ones, and how the runtime per frame compares with the video frame rate. It should also include a short analysis of failure cases, such as fruit hidden for longer than the buffer, clusters of touching fruit, and fruit entering and leaving at the image border, supported by the examples in Figures 8.6 and 8.11.

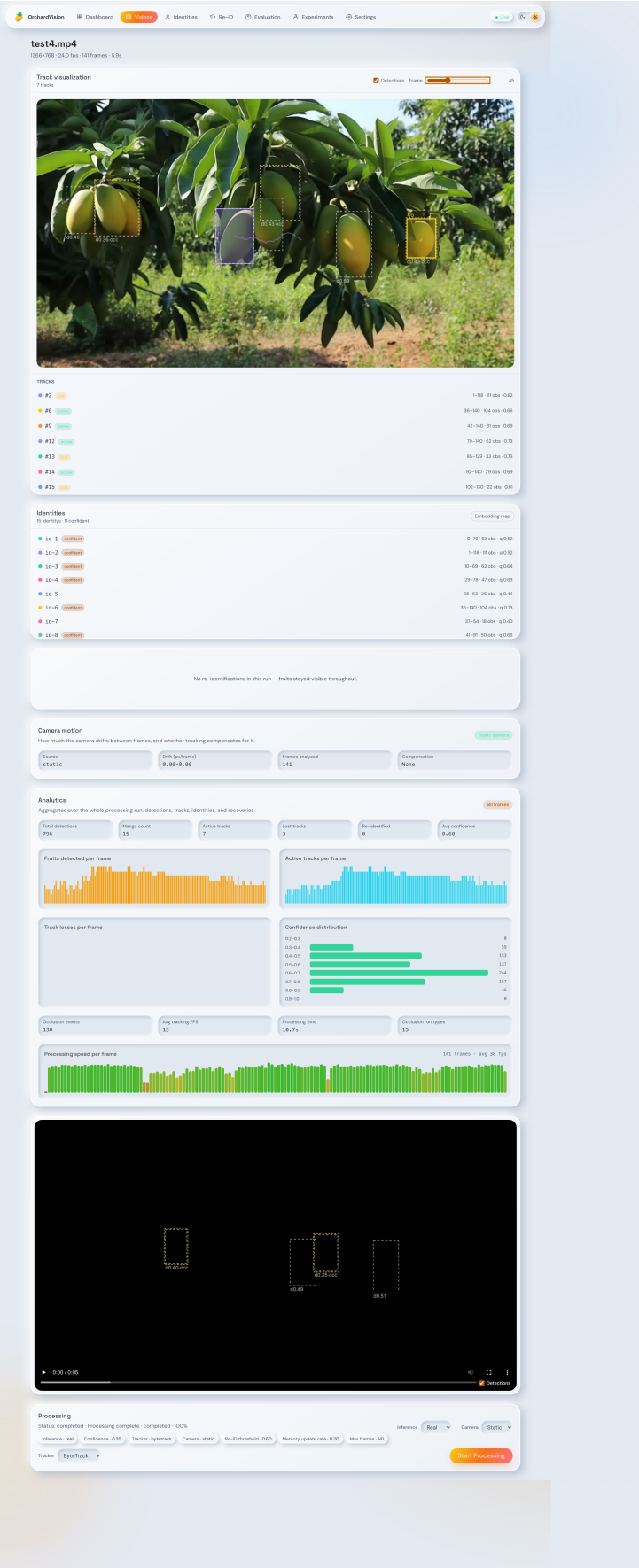


Figure 8.11: Track overlay on a processed video.

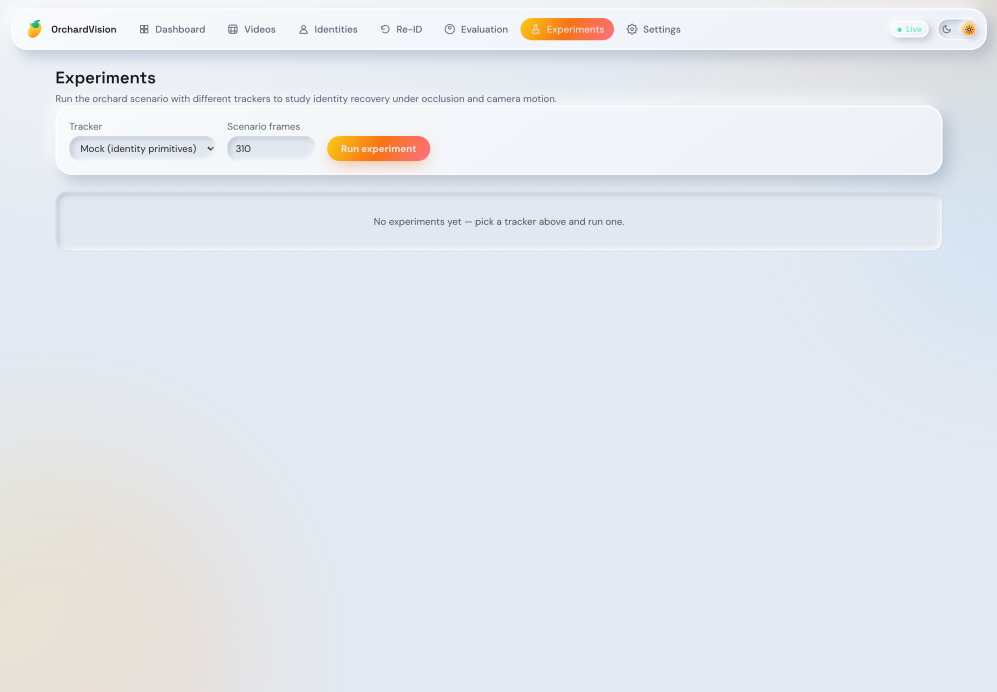


Figure 8.12: Experiment list and metrics view.

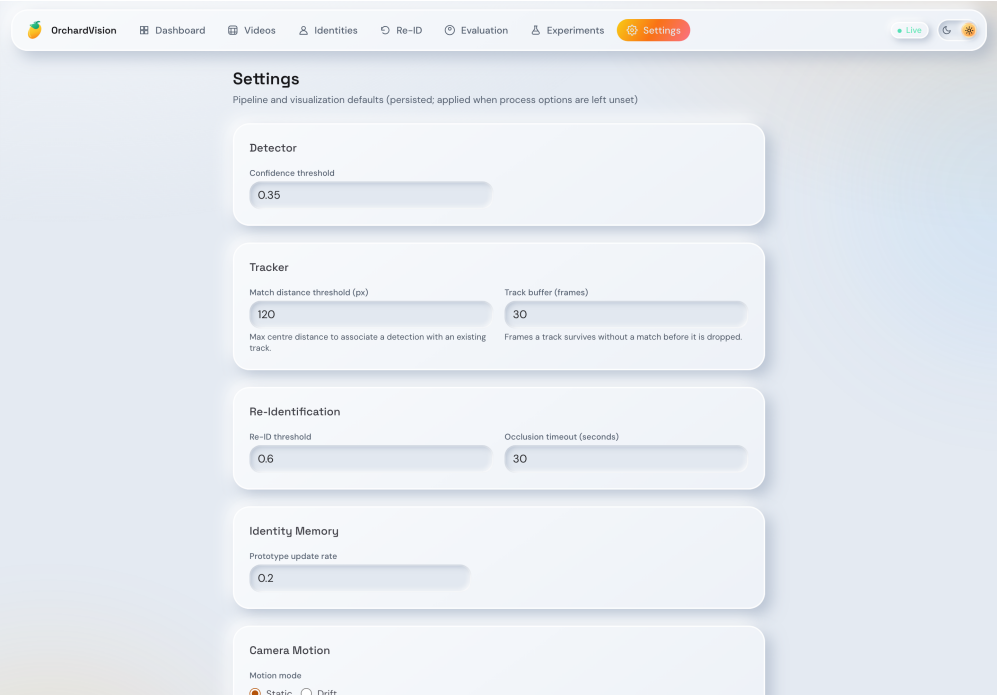


Figure 8.13: Pipeline settings screen.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 CONCLUSION

OrchardVision delivers a complete system for long-term fruit tracking and counting in mango orchards. A FastAPI backend with a SQLAlchemy 2.0 data layer runs a processing pipeline (frame extraction, detection, tracking, ReID embedding, identity consolidation and analytics), persists every intermediate and final artefact, and exposes them through a documented REST API. A React and TypeScript frontend with shared type definitions provides video management, inspection of tracks and identities, and an experiment harness that evaluates trackers with IDF1, HOTA and ID-switch metrics and exports the results as JSON or CSV.

The main take-away points are:

- identities are first-class, persistent objects (prototype, quality, observation count, confidence), which allows a fruit that disappears and reappears to be recognised rather than counted twice;
- the tracker back-end (mock, ByteTrack, TAPNext), camera-motion mode and thresholds are configurable and can be compared reproducibly through the experiment harness;
- the same code runs on a laptop with SQLite and as a container stack in which a queue worker processes the long-running jobs, so nothing about the pipeline depends on the deployment.

Measured against the objectives of Section 1.5, the pipeline, data model, API, experiment harness and deployment configuration are in place. The accuracy objectives can only be claimed once Chapter 8 has been populated with results from real orchard videos; until then the quality of the counts should be treated as unverified. A realistic limitation is that identity matching rests on appearance embeddings of visually similar fruit, so merged or fragmented identities are expected in dense clusters and after very long gaps.

9.2 FUTURE WORK

The following extensions would increase the value of the system:

- evaluate on a larger, annotated, multi-season mango dataset and publish the results;
- fine-tune the ReID network on fruit crops with a metric-learning loss and compare prototype aggregation strategies;
- replace the monocular heuristics with depth or multi-camera information to resolve fruit that occlude each other;
- link identities across recording sessions to track individual fruit through the season (the long-term setting);
- estimate fruit size and maturity for yield-weight prediction;
- add authentication and role-based access control for multi-user deployments;
- add live-stream ingestion and incremental processing so that counts update while the video is being recorded.

REFERENCES

[1]. Aharon, N., Orfaig, R., & Bobrovsky, B.-Z. (2022). BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*.

[2]. Bargoti, S., & Underwood, J. (2017). Deep fruit detection in orchards. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (pp. 3626–3633). IEEE.

[3]. Bayer, M. (2012). SQLAlchemy. In A. Brown & G. Wilson (Eds.), *The Architecture of Open Source Applications, Volume II*. aosabook.org.

[4]. Bernardin, K., & Stiefelhagen, R. (2008). Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing, 2008*, Article 246309.

[5]. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)* (pp. 3464–3468). IEEE.

[6]. Doersch, C., Gupta, A., Markeeva, L., Recasens, A., Smaira, L., Aytar, Y., Carreira, J., Zisserman, A., & Yang, Y. (2022). TAP-Vid: A benchmark for tracking any point in a video. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[7]. Hermans, A., Beyer, L., & Leibe, B. (2017). In defense of the triplet loss for person re-identification. *arXiv preprint arXiv:1703.07737*.

[8]. Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering, 82*(1), 35–45.

[9]. Koirala, A., Walsh, K. B., Wang, Z., & McCarthy, C. (2019). Deep learning – Method overview and review of use for fruit detection and yield estimation. *Computers and Electronics in Agriculture, 162*, 219–234.

[10]. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly, 2*(1–2), 83–97.

[11]. Liu, X., Chen, S. W., Liu, C., Shivakumar, S. S., Das, J., Taylor, C. J., Underwood, J., & Kumar, V. (2019). Monocular camera based fruit counting and mapping with semantic data association. *IEEE Robotics and Automation Letters, 4*(3), 2296–2303.

[12]. Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., & Leibe, B. (2021). HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision, 129*(2), 548–578.

[13]. Ramírez, S. (n.d.). *FastAPI* [Computer software]. https://fastapi.tiangolo.com

[14]. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788). IEEE.

[15]. Ristani, E., Solera, F., Zou, R., Cucchiara, R., & Tomasi, C. (2016). Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision Workshops* (pp. 17–35). Springer.

[16]. Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)* (pp. 3645–3649). IEEE.

[17]. Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)* (pp. 1–21). Springer.

[18]. Zholus, A., Doersch, C., Yang, Y., Koppula, S., Patraucean, V., He, X. O., Rocco, I., Sajjadi, M. S. M., Chandar, S., & Goroshin, R. (2025). TAPNext: Tracking any point (TAP) as next token prediction. *arXiv preprint arXiv:2504.05579*.

[19]. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)* (pp. 226–231). ACM.

[20]. Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A, 374*(2015.02052).

[21]. van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research, 9*(Nov), 2579–2605.

[22]. Reddi, R., Kreibich, G., Poffenbarger, N., & Chandra, R. (2017). A loss function for deep metric learning. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 4187–4195). IEEE.

[23]. YOLO Team & Ultralytics (2024). *Ultralytics YOLO* [Computer software]. https://github.com/ultralytics/ultralytics

APPENDIX 1: REPOSITORY LAYOUT

The directories referred to in the project documentation are shown below.

```
apps/
  api/
    app/
      main.py          # FastAPI app, router mounting, error handlers
      core/            # settings, database session, errors, logging
      models/          # SQLAlchemy 2.0 models (video, track, identity, ...)
      routers/         # health, videos, jobs, experiments, settings
      schemas/         # request / response schemas
      services/        # ProcessingService, storage, frames, analytics,
                       # embedding projection, Celery app
    alembic/           # database migrations
    tests/             # pytest suite
  web/
    src/
      api/             # typed API client and resource wrappers
      components/      # tracking, videos, landing, ui components
      hooks/           # React Query hooks per resource
      layouts/         # application shell
      pages/           # one file per route
      router/          # route table
ml/
  orchardvision_ml/    # reusable ML library
    detection/         # YOLO and mock detectors
    tracking/          # mock, ByteTrack and TAPNext trackers
    identity/          # embedders, identity dataclass, clustering
    memory/            # prototype memory (quality-aware moving average)
    reid/              # re-identification engine
    motion/            # camera-motion estimators
    evaluation/        # metrics and scenario evaluation
    pipeline/          # library-level pipeline
    camera/            # video sources
packages/
  types/               # shared TypeScript types (@orchardvision/types)
docker/                # Dockerfile and compose stack (api, worker, redis)
docs/                  # this report's LaTeX sources
data/                  # SQLite database, uploads, frame cache
models/                # YOLO and TAPNext checkpoints
scripts/               # evaluation and utility scripts
```

APPENDIX 2: RUN COMMANDS

```
# backend (development; SQLite at data/orchardvision.db)
cd apps/api && uv run uvicorn app.main:app --reload --port 8001

# frontend (Vite dev server, proxies /api/ to the backend)
cd apps/web && npm run dev

# backend tests
cd apps/api && uv run pytest

# frontend unit tests
cd apps/web && npm run test

# end-to-end tests (starts the API and the dev server)
cd apps/web && npm run test:e2e

# type-check the shared types
cd packages/types && npx tsc --noEmit

# container stack: API + Celery worker + Redis
docker compose -f docker/docker-compose.yml up --build
```

Table 7.2 refers to the phases of the project plan.

APPENDIX 3: ILLUSTRATIVE API RESPONSES

The following responses are illustrative: the field names follow the data model in Chapter 4 and the routes follow Chapter 6, while the values are examples. Note that identifiers are 32-character hexadecimal UUIDs and that the job progress is a percentage.

GET /api/jobs/{job_id}

```
{
  "job_id": "6f1c9b2ea4d84f0e9a1b2c3d4e5f6071",
  "video_id": "b83d2f6c1a4b47e9b0d5c8a7e2f10436",
  "status": "running",
  "stage": "tracking",
  "progress": 68.0,
  "message": null,
  "inference_mode": "mock",
  "config": {"tracker": "bytetrack", "confidence_threshold": 0.35}
}
```

GET /api/videos/{id}/identities/{identity_id}

```
{
  "id": "9c5e1a74b2c94d8fa0e3d6b9c8f21507",
  "video_id": "b83d2f6c1a4b47e9b0d5c8a7e2f10436",
  "identity_id": 7,
  "quality": 0.82,
  "observation_count": 143,
  "first_frame_id": 12,
  "last_frame_id": 466,
  "last_track_id": 7,
  "is_confident": true,
  "prototype": [0.021, -0.113, "..."]
}
```

GET /api/videos/{id}/reid-events

```
{
  "events": [
    {"id": "1a7c...", "track_id": 7, "victim_track_id": 7,
     "identity_id": 7, "lost_frame_id": 412,
     "reappearance_frame_id": 466, "similarity": 0.81,
     "confidence": 0.74, "accepted": true}
  ],
```

```
  "total": 1
}
```

Error response (404)

```
{"code": "video_not_found", "detail": "Video 99 not found"}
```